Designing and Implementing the Ardan IR

A deep guide to `IRBuilderVisitor`: the IR model, SSA construction, closures, and IR emission for every statement and expression

Source of truth: `ardan-lang/IR/ir/IRBuilderVisitor/IRBuilderVisitor.{hpp,cpp}`, `ardan-lang/IR/ir/IRFunction/IRFunction.hpp`, `ardan-lang/IR/ir/IRValue/IRValue.hpp`, `ardan-lang/IR/ir/IRModule/IRModule.hpp`. The guide also looks at the consumers downstream: `ardan-lang/Turbine/Turbine.cpp` and `ardan-lang/engines/DeathStar/DeathStar.cpp`.

Companion docs: `docs/compiler/03-ir-model.md`, `04-ir-builder.md`, `08-worked-example.md` and `09-known-issues.md`. Issue IDs such as **IR1**, **T5** and **V4** refer to the tables in `09-known-issues.md`. Issues that this guide finds for the first time are numbered **N1, N2, …** and collected in Appendix B.

Contents

How to read this guide

Part I — Foundations

1. Where the IR sits
2. Why have an IR at all?
3. The data model
4. The opcode set
5. The contract: invariants the builder must maintain
6. The builder's machinery

Part II — The core algorithms

7. Names, scopes and SSA values
8. SSA construction: Phi nodes
9. Capture analysis: which variables live in memory?
10. Contexts and closures: the runtime model

Part III — The `visit*` reference

11. Program entry and leaf expressions
12. Statements
13. Expressions
14. Functions, classes and advanced constructs

Part IV — Building it out

15. Recommended order of work
16. Tooling: printer, verifier, cleanup
17. Testing strategy
18. Worked example: one program, start to finish (proposed builder)

Appendix A — Status of every `visit*` method

Appendix B — Issues found by this guide

Appendix C — Further reading

How to read this guide

The guide has four parts:

Part	What it gives you
I. Foundations	Why the IR exists, what shape it has, every data structure, and the contract the builder must honour for Turbine and DeathStar to work.
II. The core algorithms	How names become SSA values, how `Phi` nodes are built, how closures and context slots work, and how capture analysis decides which variables live in memory.
III. The `visit*` reference	One section per `visit*` method in `IRBuilderVisitor.cpp`. Each covers the AST shape, what the code does today line by line, the IR it emits, its defects, and a recommended implementation with code and example output.
IV. Building it out	The order of work, a verifier and printer, a testing strategy, and complete worked programs.

Every IR listing in this guide uses the notation below:

```
function #1 'n'                   ; IRFunction index in irModule.functions, and its name
  n1:                             ; BasicBlock name (prefix + blockId)
    %2 = Constant 90              ; result = op immediate/operands
    StoreCurrentContextSlot %2 slot=0
    If %5 -> if.then3, merge.block4  ; terminator with targets
  merge.block4:  preds=[n1, if.then3]
    %9 = Phi %2, %7               ; operand k flows in from preds[k]
```

When a listing shows **today's** output, it says so. Listings marked **(proposed)** show output from the recommended implementation.

Part I — Foundations

1. Where the IR sits

```
 Source ─► Scanner ─► Parser ─► AST ─► IRBuilderVisitor ─► IRModule ─► Pharoah ─► Turbine ─►
Compiled ─► DeathStar
                                      (this guide)        (in place,   (AssemblyLine)
(bytecode)    (VM)
                                                           disabled)
```

`test_ir()` in `ardan-lang/main.cpp` drives the pipeline:

```
auto ast = get_ast(source, entryFileName);   // Scanner + Parser
IRBuilderVisitor builder;
builder.build(ast);                           // AST → IRModule
AssemblyLine assemblyLine;
Compiled compiled = assemblyLine.start(builder.irModule);   // IR → bytecode
DeathStar deathStar(compiled);
deathStar.runProgram();
```

The IR builder is a **visitor over the shared AST**. It is a sibling of the tree-walking `Interpreter` and of the older code generators (Cascade, Nova, Peregrine). It sees exactly the same nodes they do, so it must accept every AST shape the parser can produce. The parser decides what reaches the builder, not the language manual. Section 6.3 lists the nodes the parser actually builds.

2. Why have an IR at all?

The Cascade and Nova back ends generate bytecode straight from the AST. That works, but it ties three separate concerns together:

1. **Semantics**: what the program means (scoping, control flow, closures).
2. **Optimisation**: making it faster (constant folding, dead code elimination, redundancy removal).
3. **Encoding**: how a particular machine wants it (accumulator, register numbering, jump offsets).

A middle-level IR separates these concerns. The builder handles (1) once. Pharoah handles (2) on a representation that is easy to analyse. Turbine handles (3). If Turbine is ever replaced by the ARM64 back end, (1) and (2) stay the same.

What makes an IR "easy to analyse" is three properties. Ardan's IR is designed around all three.

Property	What it means	Why it matters
Explicit control flow	The code is a graph of *basic blocks*: straight-line instruction sequences joined by edges.	Loops, branches and dominance become graph questions, not tree-walking questions.
Explicit data flow (SSA)	Every value is *defined once* and named (`%7`). Uses point straight at definitions.	"Where did this value come from?" takes O(1). Constant propagation, dead code elimination and copy propagation become trivial.

Property	What it means	Why it matters
Small, uniform instructions	Each instruction does one thing (`Add`, `LoadContextSlot`, `Call`), with explicit operands.	Passes don't need to understand every AST form, only a few dozen opcodes.

2.1 Basic-block CFG versus sea of nodes

The `IROp` enum contains opcodes such as `Start`, `End`, `Region`, `Merge`, `EffectPhi`, `FrameState`, `Checkpoint` and `Projection`. These are left over from an earlier, now commented-out design: a V8 TurboFan-style **sea of nodes** in `IR/ir/IRNode`, `IRGraph` and `IRBuilder`. In a sea of nodes, instructions float freely, ordered only by data, effect and control edges.

The live design is a **basic-block CFG with SSA values**, which is much simpler. That makes it the right choice for Ardan today:

- Instructions have a fixed order inside a block, so side effects are ordered for free. No effect chain is needed.
- Lowering to an accumulator bytecode is a linear walk.
- It matches the textbooks (Cytron et al.; Braun et al. 2013, *Simple and Efficient Construction of SSA Form*), which this guide follows.

Rule: treat `Start`, `End`, `Region`, `Loop`, `Merge`, `EffectPhi`, `FrameState`, `StateValues`, `Checkpoint` and `Projection` as **reserved**. The builder must not emit them. Section 4.3 classifies every opcode.

2.2 The two homes of a variable

The most important design decision in this IR is that **a source variable lives in one of two places**:

1. **As an SSA value (a "register").** The variable is just a *name* that the builder's symbol table maps to whichever `IRValue` last defined it. Assigning `x = x + 1` emits **no store instruction**. It emits `%9 = Add %4, %8` and rebinds `x → %9`. Where control flow merges, a `Phi` picks the right definition.
2. **In a context slot ("memory").** If an inner function captures the variable, it must outlive the frame and be shared by reference. It then lives in a heap `Context` object. Every read is a `LoadContextSlot` / `LoadCurrentContextSlot`, and every write is a `StoreContextSlot` / `StoreCurrentContextSlot`. Context-allocated variables never take part in Phi construction, because memory is not SSA.

Capture analysis (§9) decides which home each variable gets. Getting that classification right is the single biggest correctness lever in the builder.

3. The data model

3.1 `IRValue`: an SSA name

```
struct IRValue {
    std::string name;   // "%7": cosmetic, for printing only
    IRType type;        // coarse static type hint
};
```

The **identity of a value is the pointer** (`shared_ptr<IRValue>`), not its name. Turbine allocates a bytecode register per *pointer*:

```
int Turbine::regFor(const std::shared_ptr<IRValue>& v) {
    auto it = registerOf.find(v.get());          // keyed by address
    ...
}
```

Consequences:

- Two `IRValue`s with the same name are **different values**.
- `createTemp()` is the **only** correct way to make a value that an instruction defines.
- A `nullptr` value is still "a value" to Turbine: all null pointers share one register, which nothing ever writes. `build()` does this today with `undefined` (§11.1).
- A value created with `make_shared<IRValue>()` but never set as an instruction's `result` is an **undefined value**. Reading it reads an uninitialised register. `visitVariable` does this for `let x;` (§12.3).

3.2 `IRType`: a hint, not a type system

```
enum class IRType { Void, I32, F64, Bool, String, Object, Function, Ptr,
                    Number, Array, Any, Null, Undefined };
```

Ardan is dynamically typed, so `IRType` is a **static approximation**. Literals get precise types (`Number`, `String`, `Bool`, `Null`, `Undefined`). Closures get `Function`. Everything whose type depends on runtime data (`Add`, `Call`, context loads) gets `Any`. Nothing downstream reads `IRType` today. It is there for a future optimiser, for example to pick `AddSmi` when both inputs are known to be `I32`.

Guidelines:

- Give a result the **most precise type you can prove** from its operands, and otherwise `Any`.
- A `Phi`'s type is the join of its operand types: equal types give that type, otherwise `Any`. Today the builder copies the type of the value before the branch, which is imprecise but harmless.
- Comparisons (`LessThan`, `StrictEqual`, …) and `Not` always produce `Bool`.

3.3 `IRInstruction`

```
struct IRInstruction {
    IROp op;
    std::vector<shared_ptr<IRValue>> operands; // SSA inputs
    std::string label;                         // free-form tag (Phi: variable name;
CallBuiltin: builtin name)
    shared_ptr<IRValue> result;                // SSA output, or nullptr for pure effects
    R immediate;                               // compile-time constant payload
    std::vector<BasicBlock*> targets;          // successor blocks for terminators
    IRFunction* childFunction = nullptr;       // Closure: which function
    int contextSlot = -1;                      // context ops: slot index; CreateContext: slot
count
    int contextDepth = 0;                      // context ops: hops up the context chain
};
```

Use each field for exactly one purpose:

Field	Use for	Never use for
`operands`	values computed at **runtime** (other instructions' results)	constants. Don't wrap a literal in a fake `IRValue` operand (see §11.6 on `True`/`False`/`StringConstant`).
`immediate`	constants known at **compile time**: number literals, string literals, property names, parameter indices	runtime values
`result`	the value the instruction defines. Always made with `createTemp()`.	values that are not freshly defined here
`targets`	control successors of a **terminator**	anything else
`label`	debugging names (Phi variable name), builtin selector for `CallBuiltin`	semantics Turbine must branch on, except `CallBuiltin`

3.4 `BasicBlock`

```
class BasicBlock {
    std::string name;
    std::vector<IRInstruction> instructions;
    std::vector<BasicBlock*> successors;
    std::vector<BasicBlock*> predecessors;
    IRInstruction* terminator = nullptr;
};
```

A basic block is a maximal straight-line sequence: control enters at the top and leaves at the bottom. Its formal invariants are in §5.

> **Pointer hazard:** `terminator` points *into* `instructions`. Any later `push_back` or `insert` on that vector can reallocate it and leave the pointer dangling. Turbine's `resolvePhis` erases from the vector, which has the same effect. **Recommendation:** treat `terminator` as derived. Compute it as `instructions.back()` whenever needed, and stop storing the pointer.

3.5 `IRFunction` and `IRModule`

```
class IRFunction { std::string name; std::vector<unique_ptr<BasicBlock>> blocks;
                   BasicBlock* entry = nullptr; bool entry_point = false; };
struct IRModule  { std::vector<unique_ptr<IRFunction>> functions; };
```

- `blocks[0]` is the function's entry block. The builder never sets `entry`, so set it in `createBlock` on the first call, or always treat `blocks[0]` as the entry.
- The **order** of `blocks` is the *layout* order Turbine lowers in. Control can fall through from `blocks[i]` to `blocks[i+1]` only if you design the lowering that way. This guide recommends an explicit terminator on every block, so layout never matters for correctness (§5).
- `irModule.functions` is filled in **post-order**: a nested function is pushed when its body finishes, so children come before parents and the program's top level (`entry_point = true`) comes last. `AssemblyLine` maps `IRFunction*` to an index, and `Closure` refers to its child by pointer (`childFunction`), so the order has no effect on meaning.

4. The opcode set

4.1 Opcode semantics

The table defines what every opcode **means**, so that the builder, the optimiser and the lowering agree. "Operands" lists runtime inputs and "Imm/fields" lists compile-time payload.

Constants

Op	Result	Operands	Imm/fields	Meaning
`Zero`	Number	–	–	the number 0
`Constant`	Number	–	`immediate`: small integer (Smi range)	an integer literal that fits in 31 bits
`HeapNumber`	Number	–	`immediate`: double	any other number
`StringConstant`	String	–	`immediate`: string	a string literal
`True` / `False`	Bool	–	–	boolean literals
`Null` / `Undefined`	Null/Undefined	–	–	the special values
`Parameter`	Any	–	`immediate`: index	the *n*-th incoming argument (proposed: index 0 = receiver/`this`)

Arithmetic and logic (pure)

Op	Operands	Meaning
`Add Subtract Multiply Divide Modulo Power`	`a, b`	JS-style arithmetic. `Add` also concatenates strings.
`Neg`	`a`	unary minus
`BitAnd BitOr BitXor ShiftLeft ShiftRight UnsignedShiftRight`	`a, b`	int32 bitwise operations
`Equal NotEqual StrictEqual StrictNotEqual LessThan LessThanOrEqual GreaterThan GreaterThanOrEqual`	`a, b`	comparisons → Bool
`Not`	`a`	`!ToBoolean(a)` → Bool
`ToBoolean ToNumber ToString`	`a`	explicit conversions
`LogicalAnd LogicalOr NullishCoalescing`	`a, b`	**Do not emit for `&&`, `||` or `??`.** These evaluate both sides, but JS short-circuits. Lower to control flow instead (§13.2). They can stay for a future optimiser to use when both sides are pure.

Memory

Op	Operands	Imm/fields	Meaning
`LoadProperty`	`obj`	`immediate`: name (string)	`obj.name`

Op	Operands	Imm/fields	Meaning
`StoreProperty`	`obj, value`	`immediate`: name	`obj.name = value`
`LoadElement`	`obj, key`	–	`obj[key]`
`StoreElement`	`obj, key, value`	–	`obj[key] = value`
`Load` / `Store`	– / `value`	`label`: slot name	a frame-local **stack slot**. Reserved for variables that cannot be SSA (variables live across `try`, §14.6).
`CreateContext`	–	`contextSlot`: slot count	make a new context whose parent is the frame's parent context, and install it as the frame's *own* context
`LoadCurrentContextSlot`	–	`contextSlot`	read a slot of the frame's own context
`StoreCurrentContextSlot`	`value`	`contextSlot`	write a slot of the frame's own context
`LoadContextSlot`	–	`contextSlot`, `contextDepth`	walk `contextDepth` hops up from the **parent** context, then read
`StoreContextSlot`	`value`	`contextSlot`, `contextDepth`	the same walk, then write

Objects, functions and calls

Op	Operands	Imm/fields	Meaning
`NewObject`	–	–	`{}`
`NewArray`	–	`immediate`: length hint	`[]`
`Closure`	`[context]`	`childFunction`	make a closure over `childFunction`
`Call`	`callee, args…`	–	call with an undefined receiver
`CallBuiltin`	`args…`	`label`: builtin name	call a runtime helper. This is the escape hatch for rare semantics (spread, iteration, `instanceof`, …).
`Print`	`args…`	–	the `print` intrinsic

Control (terminators)

Op	Operands	Targets	Meaning
`If`	`cond`	`[ifTrue, ifFalse]`	branch on `ToBoolean(cond)`
`Return`	`value`	–	return from the function
`Throw`	`value`	–	throw
`Jump` *(proposed, missing)*	–	`[target]`	unconditional branch

SSA

Op	Operands	Meaning
`Phi`	one per predecessor	selects the operand whose predecessor edge control arrived on. `label` = variable name.

4.2 Proposed additions, kept to a minimum

The IR has **no unconditional jump**. That single gap is why `if/else`, loops, `break` and `continue` cannot be expressed correctly. The design principle for any extension is:

1. Prefer an **existing** opcode, for example `Equal %x, Null` for nullish tests, or `BitXor %x, -1` for `~x`.
2. Otherwise use **`CallBuiltin` with a `label`**, for semantics that optimisation never needs to see through (spread, iterators, `instanceof`, `in`, class wiring).
3. Add a **new opcode** only when an optimiser benefits from seeing it, or when the bytecode already has a direct instruction for it.

The additions this guide recommends:

New op	Why	Lowers to (exists in `Bytecode`)
`Jump`	unconditional edges	`kJump`
`TypeOf`	common, and optimisable (`typeof x === "number"`)	`kTypeOf`
`DeleteProperty`	has a direct bytecode	`kDeletePropertyStrict`
`CallProperty`	method calls need a receiver	`kCallProperty`
`Construct`	`new`	`kConstruct`
`LandingPad`	first instruction of a `catch` block: produces the thrown value (§14.6)	handler-table entry, plus reading the accumulator on entry (needs VM support)

4.3 Status of every opcode

✅ = emitted / lowered / executed today; ✗ = not; ⚠ = present but broken.

Op	Builder emits	Turbine lowers	DeathStar runs
`Zero`	✅	⚠ extra operand (T8)	✅
`Constant`	✅	⚠ `get<short>` (T3)	⚠ unsigned (V3)
`HeapNumber`, `StringConstant`	✅	⚠ pool index off by one (T4)	✗ (V1)
`True False Null Undefined`	✅	✅	✗ (V2)
`Add Subtract Multiply`	✅	✅	✅
other binary ops	✅	✗ (T2)	✗
`LogicalAnd/Or/Nullish`	⚠ wrong semantics	✗	✗
`Neg Not ToNumber …`	✗ (IR3)	✗	✗
`Phi`	✅	✅ via copies (T6)	n/a
`If`	✅	✗ emits nothing (T1)	✗
`Return`	✅	⚠ no operand (T7)	✅
`Closure CreateContext`	✅	✅	✅

Op	Builder emits	Turbine lowers	DeathStar runs
context load/store	✅ (⚠ IR4, IR5)	✅	✅
`Call`	✅	✅	⚠ args dropped (V4)
`Print`	✅	✅	✅
`Parameter`	✗ (IR2)	✗	✗
property/element ops, `NewObject`, `NewArray`, `CallBuiltin`, `Throw`, `Load`, `Store`	✗	✗	✗

5. The contract: invariants the builder must maintain

These invariants hold the system together. Turbine relies on several of them **silently** and crashes when they are broken. §16 shows a verifier that checks all of them.

#	Invariant	Who relies on it	Broken today by
C1	Every block is **non-empty** and its **last instruction is a terminator** (`If`, `Jump`, `Return`, `Throw`), and no terminator appears anywhere else.	`Turbine::lowerBlock` lowers `instructions[0 .. size−2]`, then flushes Phi copies, then lowers `instructions.back()` as "the jump". An empty block makes `size()−1` underflow (T5).	then/else exits, loop bodies with nested flow, `visitForIn`'s orphan block, the code after `return`
C2	`successors` and `predecessors` are mirror images: `B ∈ A.successors ⇔ A ∈ B.predecessors`, with multiplicity.	Phi resolution, any dataflow pass	`visitWhile`/`visitFor` never give the header any predecessors (IR6)
C3	A block's `successors` equal its terminator's `targets`.	layout and jump lowering	loop back edges
C4	All `Phi`s sit at the **start** of their block, and a Phi has **exactly one operand per predecessor**: `operands[k]` is the value flowing in from `predecessors[k]`.	`Turbine::resolvePhis` places operand *k* in `pendingPhis[predecessors[k]]`	`visitFor` emits Phis after the `If` (IR7). Loop headers have Phis but no predecessors (IR6).
C5	Every operand is the `result` of some instruction that **dominates** the use (in the same function). For a Phi operand, the definition must dominate the end of the matching predecessor.	register allocation, any optimiser	`let x;` (undefined value), `undefined in build()`, the latch value after a loop (N2)
C6	Every SSA value belongs to **one function**. A value defined in function *F* must never be an operand in function *G*.	Turbine allocates registers per function (one `Turbine` per `IRFunction`)	`Closure` passing an outer function's context value from a non-owning function (N4)

#	Invariant	Who relies on it	Broken today by
C7	Context accesses use `Current*` with depth `contextDepth` ignored when, and only when, they target the frame's own context. Otherwise `contextDepth` counts hops from the **parent** context.	DeathStar context ops	IR4, IR5
C8	A `Closure`'s operand (if any) is the context that is current at that point in the frame.	DeathStar `kCreateClosure`	N4
C9	A function's last reachable block ends in `Return`.	VM: otherwise `ip` runs off the end (IR9)	missing implicit return

6. The builder's machinery

6.1 State

```
IRModule irModule;                      // output
IRFunction* currentFunction;            // where createBlock() adds blocks
BasicBlock* currentBlock;               // where emit() appends
int temp, blockId;                      // name counters (global across functions)
vector<Scope> scopes;                   // SSA symbol tables for the current function only
vector<Context> contexts;               // context frames of all enclosing context-owning
functions
bool currentFunctionOwnsTopContextFrame;// does contexts.back() belong to currentFunction?
```

`variables` and `allocator` are declared but never used. `createInstruction()` is defined but never called.

6.2 Helpers

```
shared_ptr<IRValue> createTemp(IRType type);         // fresh "%N"
BasicBlock* createBlock(const std::string& prefix);  // appended to currentFunction->blocks
void emit(const IRInstruction inst);                 // currentBlock->instructions.push_back
```

`visitNumericLiteral` and `visitStringLiteral` push to `currentBlock->instructions` directly instead of calling `emit()`. Route **all** emission through `emit()`. That gives one place to enforce C1, which is "don't emit after a terminator", and one place to hook an IR printer.

Recommended CFG helpers. Add these to the class. Every recommended implementation in Part III uses them.

```
// --- in IRBuilderVisitor.hpp (private) ---
static bool isTerminator(IROp op) {
    return op == IROp::If || op == IROp::Jump || op == IROp::Return || op == IROp::Throw;
}

bool isTerminated(BasicBlock* b = nullptr) const {
    b = b ? b : currentBlock;
    return !b->instructions.empty() && isTerminator(b->instructions.back().op);
}
```

```
void addEdge(BasicBlock* from, BasicBlock* to) {          // keeps C2 by construction
    from->successors.push_back(to);
    to->predecessors.push_back(from);
}

void emitJump(BasicBlock* target) {                        // C1 + C2 + C3
    if (isTerminated()) return;                            // unreachable tail: nothing to do
    IRInstruction j(IROp::Jump, nullptr, {});
    j.targets = { target };
    emit(j);
    addEdge(currentBlock, target);
}

void emitBranch(shared_ptr<IRValue> cond, BasicBlock* ifTrue, BasicBlock* ifFalse) {
    if (isTerminated()) return;
    IRInstruction br(IROp::If, nullptr, { cond });
    br.targets = { ifTrue, ifFalse };
    emit(br);
    addEdge(currentBlock, ifTrue);
    addEdge(currentBlock, ifFalse);
}

// After a terminator (return/throw/break/continue), keep emitting into a fresh,
// unreachable block so later statements do not land after the terminator.
void startUnreachableBlock() { currentBlock = createBlock("dead"); }

shared_ptr<IRValue> emitValue(IROp op, IRType type,
                              vector<shared_ptr<IRValue>> operands = {}) {
    auto dst = createTemp(type);
    emit(IRInstruction(op, dst, std::move(operands)));
    return dst;
}

shared_ptr<IRValue> emitUndefined() { return emitValue(IROp::Undefined, IRType::Undefined); }
```

Unreachable `dead` blocks have no predecessors. Delete them before lowering, either with a reachability sweep at the end of `build()` (§16) or in Pharoah. A dead block that stays empty would break C1, so the cleanup pass is required, not optional.

6.3 The return convention: `R` and `get<shared_ptr<IRValue>>`

Every visitor returns `R`, a large `std::variant` that includes `shared_ptr<IRValue>` and `bool`. The convention is:

- **Expression visitors return the `shared_ptr<IRValue>` holding the expression's value.**
- **Statement visitors return `true`.**

Callers unwrap with `get<shared_ptr<IRValue>>(expr->accept(*this))`. If an expression visitor returns `true` (every stub does, and so do `print(...)` and assignments today), the `get` throws `std::bad_variant_access`. That one exception is behind IR3, IR10, and many "random" crashes.

Recommendation:

```
shared_ptr<IRValue> IRBuilderVisitor::value(Expression* e) {
    R r = e->accept(*this);
    if (auto p = std::get_if<shared_ptr<IRValue>>(&r)) return *p;
    throw std::runtime_error("IR: expression produced no value (unimplemented visitor?)");
}
```

```
[[noreturn]] void unsupported(const char* what) {
    throw std::runtime_error(std::string("IR builder: unsupported construct: ") + what);
}
```

Use `value(e)` everywhere instead of the bare `get`. Make every not-yet-implemented visitor call `unsupported("…")` instead of `return true`. A silent `true` turns a missing feature into a confusing crash three frames away.

What the parser actually builds. These are the `make_unique<…>` calls in `Parser/`. It decides which visitors are reachable:

Reachable	**Not reachable (parser never builds it)**
`BinaryExpression`: **all** binary operators, **including `=`, compound assignments, `&&`, `||` and `??`**	`AssignmentExpression` (construction commented out at `Parser.hpp:164`)
`UnaryExpression`: `! ~ + - typeof delete void`, **and prefix `++`/`--`**	`LogicalExpression`
`UpdateExpression`: **postfix** `++`/`--` only (`prefix=false`)	`LiteralExpression`
`ConditionalExpression`, `CallExpression`, `MemberExpression`, `NewExpression`, `ArrayLiteralExpression`, `ObjectLiteralExpression`, `TemplateLiteral`, `FunctionExpression`, `ArrowFunction`, `ClassExpression`, `SequenceExpression`, `CommaExpression`, `SpreadExpression`, `AwaitExpression`, `YieldExpression`, `ThisExpression`, `SuperExpression`, keyword literals, `RestParameter`, `UIViewExpression`, modifier keywords	`PropertyExpression`

Implement `visitAssignment` and `visitLogical` by delegating to the same helpers `visitBinary` uses, so they are correct if the parser ever starts building them. Make `visitLiteral` and `visitProperty` call `unsupported`.

Part II — The core algorithms

7. Names, scopes and SSA values

7.1 The `Scope` stack

```
struct Scope {
    enum class Type { Global, Function, Block };
    Type type;
    Scope* parent;                                  // unused
    std::unordered_map<std::string, int> locals;    // unused
    unordered_map<string, shared_ptr<IRValue>> symbols;   // name → current SSA value
};
```

`scopes` is a stack of symbol tables **for the function being built**. `visitFunction` saves the outer stack, clears it, and pushes a single `Function` scope. An inner function therefore *cannot* see its parent's SSA values. That is intentional: a parent's registers don't exist in the child's frame (C6). Any name the child needs from a parent must come through a context slot.

`symbols[name]` holds the **current definition** of `name`. This is the core idea of building SSA "on the fly" (Braun et al.): instead of a later renaming pass, the builder keeps track of the reaching definition as it walks the program in order.

```
let x = 1;        →  %0 = Constant 1          scopes: { x → %0 }
x = x + 2;        →  %1 = Constant 2
                     %2 = Add %0, %1          scopes: { x → %2 }   (no store emitted)
print(x);         →  Print %2
```

7.2 `declare(name, value, kind)`: today

1. If the current function owns the top context frame **and** `name` has a slot there, it emits `StoreCurrentContextSlot value slot=N` and returns. The variable lives in memory.
2. `let` / `const`: bind in `scopes.back()`, the innermost scope.
3. `var`: bind in the innermost scope that is not a `Block` (function or global). This is the *placement* half of `var` hoisting. The *reading-before-declaration* half (reading `var x` above its declaration gives `undefined`) is not implemented.

Observations:

- `const` is never checked for re-assignment. Add a `kinds` map per scope, so that `store()` can raise "Assignment to constant variable" at compile time.
- Redeclaration (`let x; let x;` in one scope) is accepted silently. JS rejects it.
- Step 1 only checks `contexts.back()`, which is correct: a function may only *declare* into its own context.

7.3 `lookup(name)`: today

1. Search `scopes` from innermost to outermost. If found, return that SSA value.

2. Otherwise search `contexts` from the top (depth 0) down. If found, emit a load:
    - `LoadCurrentContextSlot slot` if `depth == 0 && currentFunctionOwnsTopContextFrame`
    - otherwise `LoadContextSlot slot, depth=depth`
3. Otherwise throw `"Undefined variable"`.

The load result is a **fresh temp each time**. Two reads of a captured variable produce two loads, which is correct: another closure may write the slot in between, so the reads can't be merged unless an optimiser proves nothing in between can write it.

Defect (IR5): wrong depth in an owner function. Look at the runtime model in §10. `LoadContextSlot` starts walking at the frame's **parent** context. When the current function owns `contexts.back()`, builder depth *d* (with *d* ≥ 1) is runtime depth *d* − *1*. The fix:

```
// builder depth: 0 = contexts.back()
// runtime depth: hops from frame.parentContext
int IRBuilderVisitor::runtimeDepth(int builderDepth) const {
    return currentFunctionOwnsTopContextFrame ? builderDepth - 1 : builderDepth;
}
```

7.4 `store(name, value)`: today, and why assignments vanish (IR1)

```
void IRBuilderVisitor::store(string id, shared_ptr<IRValue> reg) {
    for (depth over contexts) if found → emit StoreContextSlot depth, slot; return;
    scopes.back().symbols[id] = reg;        // ← rebinding in the INNERMOST scope
}
```

Three defects:

1. **IR1, the most damaging bug in the builder.** The rebinding goes into `scopes.back()`, not into the scope that declared `id`. Inside `if (c) { x = 5; }`, the innermost scope is the block's. `visitBlock` pops it at `}` and takes the new binding with it. The outer `x` never changes, so no Phi is built and the assignment is lost. The same thing happens in loop bodies.
2. **IR4.** The context branch always emits `StoreContextSlot`, even when the slot is in the frame's **own** context (depth 0 in an owner function), where it must emit `StoreCurrentContextSlot`.
3. **IR5.** The same depth off-by-one as `lookup`.

It also searches contexts *before* scopes, the opposite order from `lookup`. If a local `let x` shadows a captured outer `x`, the write goes to the wrong variable.

Recommended `lookup` / `store`. Both resolve the name the same way, in the same order:

```
shared_ptr<IRValue> IRBuilderVisitor::lookup(const string& name) {
    for (auto it = scopes.rbegin(); it != scopes.rend(); ++it) {
        auto f = it->symbols.find(name);
        if (f != it->symbols.end()) return f->second;
    }
    for (int d = 0; d < (int)contexts.size(); d++) {
        auto& ctx = contexts[contexts.size() - 1 - d];
        auto s = ctx.slots.find(name);
        if (s == ctx.slots.end()) continue;
        auto dst = createTemp(IRType::Any);
        bool own = (d == 0 && currentFunctionOwnsTopContextFrame);
        IRInstruction ld(own ? IROp::LoadCurrentContextSlot : IROp::LoadContextSlot, dst, {});
        ld.contextSlot = s->second;
```

```
        if (!own) ld.contextDepth = runtimeDepth(d);
        emit(ld);
        return dst;
    }
    throw std::runtime_error("ReferenceError: " + name + " is not defined");
}

void IRBuilderVisitor::store(const string& name, shared_ptr<IRValue> v) {
    // 1. SSA variable: rebind it in the scope that DECLARED it.
    for (auto it = scopes.rbegin(); it != scopes.rend(); ++it) {
        auto f = it->symbols.find(name);
        if (f != it->symbols.end()) { f->second = v; return; }
    }
    // 2. Context variable: write the slot.
    for (int d = 0; d < (int)contexts.size(); d++) {
        auto& ctx = contexts[contexts.size() - 1 - d];
        auto s = ctx.slots.find(name);
        if (s == ctx.slots.end()) continue;
        bool own = (d == 0 && currentFunctionOwnsTopContextFrame);
        IRInstruction st(own ? IROp::StoreCurrentContextSlot : IROp::StoreContextSlot, nullptr,
{ v });
        st.contextSlot = s->second;
        if (!own) st.contextDepth = runtimeDepth(d);
        emit(st);
        return;
    }
    // 3. Unknown name: JS sloppy mode creates a global; strict mode throws.
    throw std::runtime_error("ReferenceError: assignment to undeclared variable " + name);
}
```

Fixing (1) does **not** break the Phi logic. It *enables* it: the Phi code in `visitIf` compares the outer scopes before and after each branch, and that comparison only finds a difference once `store` writes to the declaring scope.

8. SSA construction: Phi nodes

8.1 The problem

```
let x = 1;
if (c) { x = 2; }
print(x);          // which definition: %x1 or %x2?
```

In SSA every value is defined once, so after the `if` the builder needs a value that means "2 if we came through *then*, else 1". That value is a **Phi**, placed at the start of the merge block:

```
entry:            %0 = Constant 1 ; ... ; If %c -> then, merge
then:   preds=[entry]   %1 = Constant 2 ; Jump merge
merge:  preds=[entry? / then]   %2 = Phi ... ; Print %2
```

The **operand order matters**. Operand *k* belongs to `predecessors[k]`. Turbine relies on exactly this: it turns every Phi into copies `operand[k] → result` placed at the end of `predecessors[k]`, just before that block's terminator. That is classic *Phi elimination by copy insertion*.

8.2 How today's code builds Phis (`visitIf`)

1. Snapshot `before = scopes`.
2. Visit *then* with `scopes = before`, and record `afterThen = scopes` and the exit block `thenExit = currentBlock`. Recording the exit block, not `thenBlock`, is correct: nested control flow inside *then* moves `currentBlock` on.
3. Do the same for *else* if there is one. Otherwise `afterElse = before` and the "else exit" is `entryBlock`.
4. `merge.predecessors = { thenExit, elseExit ?: entryBlock }`.
5. For every `(scope i, name)` in `before`: if `afterThen[i][name]` or `afterElse[i][name]` differs from `before[i][name]`, emit `Phi afterThen[i][name], afterElse[i][name]` and rebind `scopes[i][name]`.

This is sound **snapshot-and-diff** SSA construction for acyclic regions. The operand order (then, else) matches the predecessor order (thenExit, elseExit). Only names that existed *before* the `if` are considered, which is exactly right: a `let` declared inside a branch is out of scope at the merge.

Its gaps: no `Jump` from the branch exits (C1, T1), and IR1 means no difference ever shows up. Both are fixed by §6.2 and §7.4.

8.3 Generalising: `mergeScopes`

The same diff is needed at every join point: `if`, `?:`, `&&`, `||`, `??`, `switch`, and loop exits reached by `break`. Factor it out once:

```
// incoming[k] = scope stack at the end of merge->predecessors[k] (same order!)
// Precondition: currentBlock == merge, merge is still empty.
void IRBuilderVisitor::mergeScopes(BasicBlock* merge, const vector<vector<Scope>>& incoming) {
    assert(incoming.size() == merge->predecessors.size());
    if (incoming.empty()) return;                       // unreachable merge
    size_t depth = incoming[0].size();
    for (auto& in : incoming) depth = std::min(depth, in.size());
    scopes.assign(incoming[0].begin(), incoming[0].begin() + depth);

    for (size_t i = 0; i < depth; i++) {
        for (auto& [name, first] : incoming[0][i].symbols) {
            vector<shared_ptr<IRValue>> ops;
            bool differs = false;
            for (auto& in : incoming) {
                auto f = in[i].symbols.find(name);
                auto v = (f == in[i].symbols.end()) ? nullptr : f->second;
                differs |= (v != first);
                ops.push_back(v);
            }
            if (!differs) continue;
            auto dst = createTemp(joinTypes(ops));
            IRInstruction phi(IROp::Phi, dst, ops);
            phi.label = name;
            merge->instructions.push_back(phi);        // block is empty → Phis come first (C4)
            scopes[i].symbols[name] = dst;
        }
    }
}
```

Callers must snapshot `scopes` **at the moment each edge into `merge` is added**, in the same order the edges are added. Because `addEdge` appends to `predecessors`, "add an edge, push a snapshot" keeps the two lists lined up automatically.

8.4 Loops: Phis before the back edge exists

A loop header joins the *entry* edge and the *back edge*. When the builder reaches the header, the back edge's values are unknown, because the body hasn't been built yet. The standard fix (Braun et al., "incomplete Phis" on "unsealed blocks") is:

1. On entering the header, create a Phi for **every live SSA name**, with one operand (the entry value), and rebind the name to the Phi.
2. Build the condition and the body. Inside them, a read of the name returns the Phi.
3. When the back edge is added (the latch jumps to the header), **append** the name's value at the latch as the Phi's second operand.
4. Optionally, remove *trivial* Phis afterwards. A Phi is trivial when all its operands are either one value *v* or the Phi itself. Replace it with *v*.

Today's `visitWhile` does step 1 and a rough version of step 3. What's missing:

- The header has **no predecessors** (IR6). Turbine indexes `predecessors[k]` for every Phi operand *k*, so it reads out of bounds and segfaults.
- The entry block never jumps to the header, and the "back edge" is an `If` on the **stale** first condition, with targets `{merge, body}`.
- The back edge is attached to `bodyBlock`, not to the block where the body **ends** (`currentBlock`). If the body has an `if` inside, they are different blocks.
- **N2 (new):** after the loop, `currentBlock = mergeBlock`, but `scopes` still holds the **latch** values. The exit is reached from the header, so the names must be bound to the **header Phis**. Otherwise, code after a zero-iteration loop reads a value whose definition never ran (C5). This only shows up once IR1 is fixed.

Section 12.7 gives the full corrected `visitWhile`. `visitFor` and `visitDoWhile` are variations of it.

8.5 Trivial-Phi cleanup

Step 1 creates a Phi for *every* visible name, including ones the loop never touches. Those Phis come out as `%p = Phi %v, %p`. They are harmless (Turbine just emits redundant copies), but they clutter the IR. Add a post-pass that replaces each trivial Phi's uses with its single real operand, and repeat until nothing changes. It belongs either at the end of `visitFunction` or as Pharoah's first pass.

```
// returns the replacement value, or nullptr if the Phi is not trivial
shared_ptr<IRValue> trivialPhiValue(const IRInstruction& phi) {
    shared_ptr<IRValue> same;
    for (auto& op : phi.operands) {
        if (op == same || op == phi.result) continue;
        if (same) return nullptr;     // two distinct real operands
        same = op;
    }
    return same;                       // nullptr only for an unreachable Phi
}
```

9. Capture analysis: which variables live in memory?

9.1 What it must compute

For each function *F*, and for each binding *b* that *F* declares (parameters, `var`/`let`/`const`, inner function names, class names, catch parameters), the question is:

> Is *b* referenced from inside any function nested in *F*, at any depth?

If it is, *b* is **captured**. *F* emits `CreateContext` with one slot per captured binding, and every access to *b* goes through that slot.

9.2 What the code computes today

```
visitFunction(F):
  vars         = collectDeclaredNames(F.body)     // top-level VariableStatements of F's block
only
  nested_vars  = collectNestedVariables(F.body)   // for each DIRECT child FunctionDeclaration
C:
                                                  //    freeVariablesOf(C.body)
  captured     = vars ∩ nested_vars
```

`freeVariablesOf` walks a body with `walkForFreeVars`, which only understands:

- `VariableStatement` (its initialiser expression, then binds the name)
- `ReturnStatement` (its argument)
- `IdentifierExpression` and `BinaryExpression`, among expressions

Gaps (IR8, plus some new ones):

Missed	Example	Result
expression statements	`function u(){ print(a); }`	`a` not captured → `Undefined variable` inside `u`
call arguments, members, arrays, objects, unary, conditional, template…	`return f(a)`	same
nested blocks, `if`, loops in the child	`if (k) { return a; }`	same
grandchildren	`function n(){ let a; function m(){ function u(){ return a; } } }`	`a` not captured
parameters as captured bindings	`function n(a){ return function(){ return a; }; }`	params aren't declared at all (IR2)
inner function names	`function n(){ function a(){} function b(){ return a(); } }`	`a` is an SSA value of `n`, so it's invisible to `b`
function expressions and arrows as children	`let f = () => a;`	only `FunctionDeclaration` children are inspected

Missed	Example	Result
shadowing	`function u(){ let a = 1; return a; }`	handled correctly, because the name is bound before later uses

Also, `walkForFreeVars` checks each identifier with `for (auto& s : boundStack) if (!s.count(name)) freeVars.insert(name)`. That reads "free if **any** level lacks it", but it should be "free if **no** level has it". Only one level is ever pushed, so this doesn't misbehave yet, but it will once nested blocks push levels.

9.3 Recommended design: a resolver pre-pass

Capture analysis is a *whole-function* property, so compute it **before** IR building, with a dedicated resolver. V8, SpiderMonkey and Roslyn all do it this way:

```
struct FunctionInfo {
    unordered_set<string> declared;          // every binding the function declares
    unordered_set<string> captured;          // subset referenced by nested functions
    vector<string>        slotOrder;         // deterministic slot assignment
    bool usesThis = false, isArrow = false;
};

class ScopeResolver {
public:
    unordered_map<const void*, FunctionInfo> info;   // keyed by
FunctionDeclaration*/FunctionExpression*/ArrowFunction*/nullptr(top level)

    void resolveProgram(const vector<unique_ptr<Statement>>& program);

private:
    struct Frame { const void* fn; vector<unordered_set<string>> blocks; };
    vector<Frame> stack;

    void declare(const string& n) { stack.back().blocks.back().insert(n);
                                    info[stack.back().fn].declared.insert(n); }

    void reference(const string& n) {
        for (int f = (int)stack.size() - 1; f >= 0; f--)
            for (auto b = stack[f].blocks.rbegin(); b != stack[f].blocks.rend(); ++b)
                if (b->count(n)) {
                    if (f != (int)stack.size() - 1)            // found in an OUTER function
                        info[stack[f].fn].captured.insert(n);
                    return;
                }
        // unresolved → global
    }

    void enterFunction(const void* fn, const vector<string>& params);
    void exitFunction() {                                      // fix the slot order
        auto& fi = info[stack.back().fn];
        fi.slotOrder.assign(fi.captured.begin(), fi.captured.end());
        std::sort(fi.slotOrder.begin(), fi.slotOrder.end()); // deterministic dumps
        stack.pop_back();
    }
    void walk(Statement*);   // full recursive walk over EVERY node kind
    void walk(Expression*);
};
```

Key rules for `walk`:

- **Hoisting:** before walking a block's statements, `declare` every `var` name (into the function's block) and every `FunctionDeclaration` / `ClassDeclaration` name in the block. Uses earlier in the block then resolve correctly.
- `IdentifierExpression` → `reference(name)`.
- An assignment whose left side is an identifier → `reference(name)` too, because assignment is a use of the binding.
- `FunctionDeclaration`, `FunctionExpression`, `ArrowFunction`, `MethodDefinition` → `enterFunction` with the parameter names, walk the body, `exitFunction`.
- `ThisExpression` inside an arrow → `reference("this")`. In a normal function, `declare("this")` on entry. Arrow functions then capture `this` lexically through the ordinary closure mechanism (§14.3).
- `CatchClause` declares `param` in the catch block. `ForStatement` / `ForIn` / `ForOf` declare their `let`/`const` in a block wrapping the loop.

`visitFunction` then just asks `info[stmt]` for `captured` and `slotOrder`, and the hand-rolled `collect*` / `walkForFreeVars` helpers go away.

9.4 Slot assignment

Today slots are numbered in `unordered_set` iteration order. That's stable within one run, but arbitrary, and it changes between standard library implementations. Sort the names, as above, so IR dumps and golden tests are reproducible.

10. Contexts and closures: the runtime model

10.1 What DeathStar does

```
struct Context  { shared_ptr<Context> parent; vector<Value> slots; };
struct Closure  { int functionIndex; shared_ptr<Context> capturedContext; };
struct CallFrame{ ...; shared_ptr<Context> ownContext; shared_ptr<Context> parentContext; };
```

Bytecode	Semantics
`kCreateClosure fn, ctxReg`	`closure.capturedContext = (ctxReg == -1) ? null : registers[ctxReg]`
call of a closure	new frame with `parentContext = closure.capturedContext`, `ownContext = null`
`kCreateFunctionContext n`	`ownContext = new Context(n, parent = frame.parentContext)`
`kLdaCurrentContextSlot s` / `kStaCurrentContextSlot s`	`ownContext->slots[s]`
`kLdaContextSlot d, s` / `kStaContextSlot d, s`	`c = parentContext; repeat d: c = c->parent; c->slots[s]`

So at runtime the chain is: **own context (if the function has captured variables) → parent context = the context current where the closure was created → its parent → …**

10.2 Mapping the builder's `contexts` stack onto the runtime chain

`contexts` holds one entry per **enclosing function that owns a context**, innermost last. Functions that don't own a context add nothing. At runtime they add nothing either, since they pass their `parentContext` through unchanged. The two chains therefore line up exactly, apart from the "own" offset:

Builder `d` (0 = `contexts.back()`)	Current function owns top?	Runtime access
0	yes	`*CurrentContextSlot`
d ≥ 1	yes	`*ContextSlot depth = d − 1`
d ≥ 0	no	`*ContextSlot depth = d`

That is the `runtimeDepth` rule from §7.3.

10.3 Which context a closure captures (N4)

Today:

```
if (!contexts.empty()) closureIr.operands.push_back(contexts.back().contextValue);
```

`contextValue` is the `%N` that `CreateContext` produced, and it is **a register of the function that owns that context**. This is correct only when the closure is created *in the owning function itself*. If the closure is created in a function *G* that owns no context but is nested in an owner *F*, the operand is *F*'s register. That breaks C6: in *G*'s frame, that register is unrelated, uninitialised, or another value, and `any_cast` throws or picks up garbage.

```
function f() { let a = 1;            // f owns a context (once capture analysis sees a)
  function g() {                      // g owns nothing
    function h() { return a; }        // Closure for h is emitted in g with operand = f's %ctx
    return h;
  }
  return g;
}
```

Recommended fix: a closure always captures **the frame's current context**, meaning `ownContext` if set and otherwise `parentContext`. That is what V8 does with its implicit context register. Then `Closure` needs **no operand**:

- IR: `Closure` keeps only `childFunction`.
- Turbine: `kCreateClosure fn` with no context operand.
- DeathStar: `closure->capturedContext = frame.ownContext ? frame.ownContext : frame.parentContext;`

If changing the bytecode is not an option, add an IR op that produces the current context (`%c = CurrentContext`, lowered to a new `kLdaCurrentContext`), and pass that `%c` as the operand.

10.4 Context lifetime and loops

`let` inside a loop body is per-iteration in JS:

```
let fs = [];
for (let i = 0; i < 3; i++) fs.push(() => i);   // 0, 1, 2, not 3, 3, 3
```

If `i` is captured, every iteration needs a **fresh** context. Function-level `CreateContext` can't provide that. The bytecode has `kCreateBlockContext` for this purpose. The recommended path: in v1, allocate per-iteration captured variables in a block context created at the top of the loop body. Until then, document the divergence: all iterations share one slot.

Part III — The `visit*` reference

Every method in `IRBuilderVisitor.cpp` is covered below. Each entry follows the same template:

- **AST:** the node's fields, as the parser fills them.
- **Today:** what the code does, step by step.
- **Emits (today):** a real IR listing, where the method emits anything.
- **Defects:** linked to issue IDs.
- **Design / Recommended implementation:** code written against the real types, plus the helpers from §6.2, §7.4 and §8.3.
- **Emits (proposed):** the IR the recommended implementation produces.

11. Program entry and leaf expressions

11.1 `build(program)`

Today

```
make IRFunction (entry_point = true); currentFunction = it
currentBlock = createBlock("entry")
scopes.push_back(Global)
for each statement: s->accept(*this)
declare("undefined", {}, Var)                // binds "undefined" → nullptr
Return lookup("undefined")                   // Return <nullptr>
scopes.pop_back(); irModule.functions.push(entry)
```

Defects

- The return value is a **null `IRValue`** (C5). Turbine gives it a register that is never written. Emit a real `Undefined` instead.
- The function is never given a `name` (it's empty in dumps) and never sets `entry`.
- **N6 (new): top-level bindings are invisible to functions.** The top level never runs capture analysis and never creates a context, so `let one = 1; function f(){ return one; }` throws `Undefined variable` while building `f`. The top level must be treated as a function: run the resolver on it and emit `CreateContext` for its captured bindings.
- No hoisting: `f(); function f(){}` fails, because `f` is declared only when its statement is reached.
- `Return` is emitted even if the last statement already terminated the block.

Recommended implementation

```
void IRBuilderVisitor::build(const vector<unique_ptr<Statement>>& program) {
    resolver.resolveProgram(program);                       // §9.3

    auto entry = make_unique<IRFunction>();
    entry->name = "<main>";
    entry->entry_point = true;
    currentFunction = entry.get();
    currentBlock = createBlock("entry");
    currentFunction->entry = currentBlock;

    scopes.push_back({ Scope::Type::Global, nullptr, {} });
```

```
    openFunctionContext(resolver.info[nullptr]);          // CreateContext if top level has
captures

    hoistFunctionDeclarations(program);                    // §14.1
    for (auto& s : program)
        if (!dynamic_cast<FunctionDeclaration*>(s.get())) s->accept(*this);

    if (!isTerminated()) {
        IRInstruction ret(IROp::Return, nullptr, { emitUndefined() });
        emit(ret);
    }
    closeFunctionContext();
    scopes.pop_back();
    removeUnreachableBlocks(*currentFunction);             // §16
    irModule.functions.emplace_back(std::move(entry));
}
```

11.2 `visitExpression(ExpressionStatement*)`

AST: `expression`.

Today: `return stmt->expression->accept(*this);`. This leaks the expression's `R` (sometimes an `IRValue`, sometimes `true`) out of a *statement* visitor.

Recommended: evaluate for side effects, throw the value away, and return `true`:

```
R IRBuilderVisitor::visitExpression(ExpressionStatement* stmt) {
    stmt->expression->accept(*this);     // the value, if any, is dead; DCE removes pure
leftovers
    return true;
}
```

An expression statement like `1 + 2;` leaves behind an unused `Add`. Nothing should special-case that, because removing it is Pharoah's job. Pharoah can only do that safely once it knows which ops have side effects (O2).

11.3 `visitNumericLiteral`

AST: `R value` (the parsed number, held in a C++ type that matches its `NumberType`), and `NumberType type` (`NONE`, `SHORT`, `INT`, `LONG`, ..., `FLOAT`, `DOUBLE`, `LONG_DOUBLE`).

Today

```
dst = createTemp(Number)
INT family: value == 0 → Zero; IsSmi(value) → Constant; else → HeapNumber
FLOAT family → HeapNumber
NONE → Zero
inst.immediate = expr->value          // the raw R variant, whatever C++ type it holds
currentBlock->instructions.push_back(inst)      // bypasses emit()
```

`IsSmi` checks the V8-style 31-bit range [-2^{30}, $2^{30}-1$]. That's the right *IR-level* idea: small integers are cheap immediates and everything else is boxed.

Defects

- The `immediate` keeps whatever C++ type the parser picked (`short`, `int`, `long`, …). Turbine reads it with `get<short>`, so any `Constant` parsed as `int` (such as `100000`) aborts (T3). **Normalise at the IR boundary:** `Constant` carries `int32_t`, and `HeapNumber` carries `double`.
- `NONE → Zero` gives the value 0 to any literal the parser couldn't classify. It should be an error, or at least `HeapNumber` with the converted value.
- `0.0` (a `DOUBLE`) becomes `HeapNumber` instead of `Zero`. That's harmless, but inconsistent.
- `-0` would become `Zero` if negative literals were folded. JS's `-0` is not `+0`, so keep `-0` as `HeapNumber`.

Recommended implementation

```
R IRBuilderVisitor::visitNumericLiteral(NumericLiteral* expr) {
    double d = toValue(expr->value).numberValue;
    bool integral = std::isfinite(d) && std::trunc(d) == d && !(d == 0 && std::signbit(d));
    auto dst = createTemp(IRType::Number);
    if (integral && d == 0) {
        emit(IRInstruction(IROp::Zero, dst, {}));
    } else if (integral && IsSmi((int64_t)d)) {
        IRInstruction c(IROp::Constant, dst, {});
        c.immediate = static_cast<int>(d);            // canonical: int
        emit(c);
    } else {
        IRInstruction h(IROp::HeapNumber, dst, {});
        h.immediate = d;                              // canonical: double
        emit(h);
    }
    return dst;
}
```

Turbine then reads `get<int>`. `kLdaSmi` must sign-extend its operand (V3).

11.4 `visitStringLiteral`

AST: `text`.

Today: emits `StringConstant` with `immediate = text`, **and** an operand that is a brand-new `IRValue` *named after the string's text*.

Defect: that operand is never defined by any instruction (C5), and its name collides with real temps in dumps (`%"hello"`). Turbine happens to ignore operands for `StringConstant`, but a verifier or DCE pass would see a use of an undefined value.

Recommended:

```
R IRBuilderVisitor::visitStringLiteral(StringLiteral* expr) {
    auto dst = createTemp(IRType::String);
    IRInstruction s(IROp::StringConstant, dst, {});
    s.immediate = expr->text;
    emit(s);
    return dst;
}
```

Optionally *intern* string constants per function, with an `unordered_map<string, shared_ptr<IRValue>>` cleared at function entry, so that repeated `"x"` literals share one value. That's only safe inside a single block, or in a block that dominates every use, so the simple version (one instruction per literal) is the right

default. Pharoah can do global value numbering later.

11.5 `visitIdentifier`

AST: `name`, `token`.

Today: `return lookup(expr->token.lexeme);`

Defects

- It uses `token.lexeme`, while `emitAssignment` uses `ident->name`. Both are filled by `IdentifierExpression(Token)`, but the `IdentifierExpression(const string&)` constructor leaves `token` empty. Always use `name`.
- There's no `undefined` handling. Parsing `undefined` produces `UndefinedKeyword`, so this is fine, but any global the program never declares throws at **compile time**. JS would only throw a `ReferenceError` at runtime, if the code actually runs. Throwing at compile time is acceptable for a v1 compiler, but say so in the docs.

Recommended: `return lookup(expr->name);`. All the substance lives in `lookup` (§7.4).

11.6 Keyword literals: `visitTrueKeyword`, `visitFalseKeyword`, `visitNullKeyword`, `visitUndefinedKeyword`

Today: `True` / `False` emit an extra operand: a fake `IRValue` named `"1"` or `"0"`, of type `Number`. `Null` / `Undefined` are clean.

Defect: the same fake-operand problem as §11.4.

Recommended:

```
R IRBuilderVisitor::visitTrueKeyword(TrueKeyword*)          { return emitValue(IROp::True,
IRType::Bool); }
R IRBuilderVisitor::visitFalseKeyword(FalseKeyword*)        { return emitValue(IROp::False,
IRType::Bool); }
R IRBuilderVisitor::visitNullKeyword(NullKeyword*)          { return emitValue(IROp::Null,
IRType::Null); }
R IRBuilderVisitor::visitUndefinedKeyword(UndefinedKeyword*){ return emitUndefined(); }
```

11.7 Unreachable nodes: `visitLiteral`, `visitProperty`

The parser never builds `LiteralExpression` or `PropertyExpression` (§6.3). `visitLiteral` returns `true` today. Both should call `unsupported(...)`, so that a future parser change fails loudly instead of crashing in `get<>`.

12. Statements

12.1 `visitBlock`

AST: `body` (statements) and `standalone`.

Today: push a `Block` scope, visit each statement, pop. That's correct.

Recommended additions

```
R IRBuilderVisitor::visitBlock(BlockStatement* stmt) {
    scopes.push_back({ Scope::Type::Block, nullptr, {} });
    hoistFunctionDeclarations(stmt->body);                // block-level function hoisting
    for (auto& s : stmt->body) {
        if (dynamic_cast<FunctionDeclaration*>(s.get())) continue;   // already emitted
        s->accept(*this);
    }
    scopes.pop_back();
    return true;
}
```

Statements after a `return`, `break`, `continue` or `throw` still get visited. They land in the unreachable block that `startUnreachableBlock()` opened, and §16 removes them. Visiting them anyway, rather than skipping, keeps compile-time errors (such as undefined names) consistent with an interpreter that checks lazily. Skipping them would be equally valid.

12.2 `visitEmpty`, `visitInterfaceDeclaration`

`;` and `interface …` emit nothing. `return true` is **correct** for both. Interfaces are purely type-level: the interpreter also ignores them.

12.3 `visitVariable`

AST: `kind` (`"var"`/`"let"`/`"const"`, possibly upper case) and `declarations` (each has an `id` string and an optional `init`).

Today

```
for each declarator:
    value = init ? get<IRValue>(init->accept) : make_shared<IRValue>()   // ← undefined value
    declare(id, value, kind)
```

Emits (today) for `let a = 90;` inside a function that captures `a`:

```
%2 = Constant 90
StoreCurrentContextSlot %2 slot=0
```

When `a` is not captured, the same source emits only `%2 = Constant 90` and binds `a → %2`.

Defects

- `let x;` binds `x` to a value no instruction defines (C5). Reading `x` reads an uninitialised register.
- `var x;` with no initialiser must **not** reset an existing `x`. `var x = 1; var x; print(x)` prints `1` in JS.
- Destructuring is impossible: `VariableDeclarator::id` is a `string`, so the parser doesn't support it either.
- `const` without an initialiser should be a compile error. The interpreter reports "Missing initializer in const declaration".

Recommended implementation

```
R IRBuilderVisitor::visitVariable(VariableStatement* stmt) {
    BindingKind kind = getBindingKind(stmt->kind);
    for (auto& decl : stmt->declarations) {
        if (!decl.init && kind == BindingKind::Const)
            throw std::runtime_error("SyntaxError: Missing initializer in const declaration");

        if (!decl.init && kind == BindingKind::Var && isDeclaredInFunctionScope(decl.id))
            continue;                                  // `var x;` re-declaration is a no-op

        auto v = decl.init ? value(decl.init.get()) : emitUndefined();
        declare(decl.id, v, kind);
    }
    return true;
}
```

Emits (proposed) for `let x; let y = x;`:

```
%0 = Undefined
```

There's no copy instruction for `y = x`: `y` is bound straight to `%0`. That's SSA's free copy propagation at work.

12.4 `visitIf`

AST: `test`, `consequent`, and `alternate` (may be null). Bodies must be braced (P1).

Today: see §8.2 for the algorithm. It creates `if.then`, optionally `if.else`, and `merge.block`. It pushes `If cond → [then, else|merge]` into the entry block, visits both arms on copies of `scopes`, sets `merge.predecessors`, and emits Phis for names that changed.

Emits (today) for `let x = 1; if (x < 2) { x = 5; } print(x);`:

```
function #0 ''
  entry0:
    %0 = Constant 1
    %2 = Constant 2
    %1 = LessThan %0, %2               ; result temp is allocated before the operands (§13.1)
    If %1 -> if.then1, merge.block2
  if.then1:                            ; no terminator (C1)
    %3 = Constant 5                    ; x = 5 rebinds in the popped block scope (IR1)
  merge.block2:  preds=[if.then1, entry0]
    Print %0                           ; no Phi → prints 1
    Return <null>
```

Turbine also lowers `If` to nothing (T1), so both arms always run.

Defects: C1 (no `Jump` out of the arms), IR1 (no Phi ever forms), T1 downstream. The merge block is created even when both arms return, so it's then unreachable. Today's predecessor order is `[thenExit, entry]`, which matches the Phi operand order `[then, else]`. It stays correct only as long as both orders are maintained together, so build the predecessor list with `addEdge` and the operand list with snapshots in edge order.

Recommended implementation

```
R IRBuilderVisitor::visitIf(IfStatement* stmt) {
    auto cond = value(stmt->test.get());

    BasicBlock* thenB = createBlock("if.then");
    BasicBlock* elseB = stmt->alternate ? createBlock("if.else") : nullptr;
    BasicBlock* endB  = createBlock("if.end");

    vector<Scope> before = scopes;
    vector<vector<Scope>> incoming;                    // parallel to endB->predecessors

    emitBranch(cond, thenB, elseB ? elseB : endB);
    if (!elseB) incoming.push_back(before);            // edge #0 into endB: the false edge

    currentBlock = thenB; scopes = before;
    stmt->consequent->accept(*this);
    if (!isTerminated()) { incoming.push_back(scopes); emitJump(endB); }

    if (elseB) {
        currentBlock = elseB; scopes = before;
        stmt->alternate->accept(*this);
        if (!isTerminated()) { incoming.push_back(scopes); emitJump(endB); }
    }

    currentBlock = endB;
    if (incoming.empty()) { scopes = before; return true; }   // both arms left; endB is dead
    mergeScopes(endB, incoming);
    return true;
}
```

Emits (proposed)

```
  entry0:
    %0 = Constant 1
    %1 = Constant 2
    %2 = LessThan %0, %1
    If %2 -> if.then1, if.end2
  if.then1:  preds=[entry0]
    %3 = Constant 5
    Jump if.end2
  if.end2:  preds=[entry0, if.then1]
    %4 = Phi %0, %3          ; x
    Print %4
    %5 = Undefined
    Return %5
```

Turbine lowering then places `%0 → %4` before the `If` in `entry0`, and `%3 → %4` before the `Jump` in `if.then1`.

> **Copy placement subtlety.** `entry0` ends in a *conditional* branch with two successors, and only the edge into `if.end2` should carry the copy `%0 → %4`. Turbine emits the copy on *both* outgoing paths. That's harmless here because nothing on the `if.then1` path reads `%4` before it is overwritten. In general, though, this is the **critical edge** problem. The standard fix is to *split critical edges* (an edge from a block with several successors into a block with several predecessors): insert an empty block on the edge that holds just the copy and a `Jump`. Do it as a pre-pass in Turbine, or have `emitBranch` create the split blocks when the target already has (or will get) another predecessor.

12.5 `visitReturn`

AST: `argument` (may be null).

Today: `Return [arg]`. With no argument, the operand list is empty (T7).

Defects: an empty `Return`. Any code that follows lands after the terminator (C1).

Recommended

```
R IRBuilderVisitor::visitReturn(ReturnStatement* stmt) {
    auto v = stmt->argument ? value(stmt->argument.get()) : emitUndefined();
    // if inside try/finally, inline the pending finalizers first (§14.6)
    IRInstruction ret(IROp::Return, nullptr, { v });
    emit(ret);
    startUnreachableBlock();
    return true;
}
```

12.6 Loop infrastructure

All loops and `switch` share one target stack:

```
struct JumpTarget {
    BasicBlock* breakTarget;
    BasicBlock* continueTarget;          // nullptr for switch
    size_t scopeDepth;                   // scopes.size() at entry; snapshots are trimmed to
this
    vector<vector<Scope>> breakScopes;   // one per break edge, in edge order
    vector<vector<Scope>> continueScopes;
};
vector<JumpTarget> jumpTargets;

vector<Scope> trimmedScopes(size_t depth) const {
    return vector<Scope>(scopes.begin(), scopes.begin() + std::min(depth, scopes.size()));
}

struct OpenPhi { size_t scope; string name; size_t index; };

// Create a 1-operand Phi for every visible SSA name and rebind it (§8.4 step 1).
vector<OpenPhi> openLoopHeader(BasicBlock* header) {
    vector<OpenPhi> open;
    for (size_t i = 0; i < scopes.size(); i++)
        for (auto& [name, v] : scopes[i].symbols) {
            auto dst = createTemp(v ? v->type : IRType::Any);
            IRInstruction phi(IROp::Phi, dst, { v });
            phi.label = name;
            header->instructions.push_back(phi);
            open.push_back({ i, name, header->instructions.size() - 1 });
            scopes[i].symbols[name] = dst;
        }
    return open;
}

// Append one operand per back edge, in the order those edges were added (§8.4 step 3).
void closeLoopHeader(BasicBlock* header, const vector<OpenPhi>& open,
                     const vector<vector<Scope>>& backEdges) {
    for (auto& p : open)
        for (auto& s : backEdges)
```

```
            header->instructions[p.index].operands.push_back(s[p.scope].symbols.at(p.name));
}
```

Header Phi indices stay valid: only Phis go in at the front, and later instructions are only appended after them.

12.7 `visitWhile`

AST: `test` and `body`.

Today (also see §8.4): it creates `while.header`, `while.body` and `merge.block`, and gives entry a successor edge but no jump. In the header it creates Phis, then the condition, then `If`. It visits the body, then appends `If <stale cond> -> merge, body` to `bodyBlock` (not to the block where the body ends) and patches the Phi operands from `scopes`. It sets `merge.predecessors = {header}` and leaves `scopes` holding the latch values.

Emits (today) for `let i = 0; while (i < 3) { i = i + 1; } print(i);`:

```
  entry0:                               ; no Jump to the header; falls into it by layout
    %0 = Zero
  while.header1:  preds=[]              ; IR6: Phis with no predecessors
    %1 = Phi %0, %1                     ; i: body's store went to the popped scope (IR1), so
self-loop
    %3 = Constant 3
    %2 = LessThan %1, %3
    If %2 -> while.body2, merge.block3
  while.body2:  preds=[while.header1]
    %5 = Constant 1
    %4 = Add %1, %5                     ; assigned to i, then lost
    If %2 -> merge.block3, while.body2  ; stale condition, wrong targets
  merge.block3:  preds=[while.header1]
    Print %1
```

Turbine segfaults in `resolvePhis` because it indexes `predecessors[0]` of the header.

Recommended implementation

```
R IRBuilderVisitor::visitWhile(WhileStatement* stmt) {
    BasicBlock* header = createBlock("while.header");
    BasicBlock* body   = createBlock("while.body");
    BasicBlock* exit   = createBlock("while.exit");

    emitJump(header);                                   // preheader → header  (header pred #0)
    currentBlock = header;
    size_t depth = scopes.size();
    auto open = openLoopHeader(header);

    auto cond = value(stmt->test.get());                // may create blocks (&&, ?:)
    vector<Scope> headerScopes = scopes;                // the state on the exit edge
    emitBranch(cond, body, exit);                       // exit pred #0

    jumpTargets.push_back({ exit, header, depth, {}, {} });
    currentBlock = body;
    stmt->body->accept(*this);

    vector<vector<Scope>> backEdges = jumpTargets.back().continueScopes;   // continue edges,
in order
    if (!isTerminated()) { backEdges.push_back(trimmedScopes(depth)); emitJump(header); }
```

```
    closeLoopHeader(header, open, backEdges);

    JumpTarget t = std::move(jumpTargets.back()); jumpTargets.pop_back();

    currentBlock = exit;
    vector<vector<Scope>> exitIn = { headerScopes };
    for (auto& b : t.breakScopes) exitIn.push_back(b);
    mergeScopes(exit, exitIn);                          // fixes N2: names bind to the header
Phis
    return true;
}
```

Emits (proposed)

```
  entry0:
    %0 = Zero
    Jump while.header1
  while.header1:  preds=[entry0, while.body2]
    %1 = Phi %0, %5          ; i
    %2 = Constant 3
    %3 = LessThan %1, %2
    If %3 -> while.body2, while.exit3
  while.body2:  preds=[while.header1]
    %4 = Constant 1
    %5 = Add %1, %4
    Jump while.header1
  while.exit3:  preds=[while.header1]
    Print %1
    %6 = Undefined
    Return %6
```

12.8 `visitDoWhile`

AST: `body` and `condition`.

Today: a stub that returns `true`. The whole loop is silently skipped.

Design: the body comes first, so the **body block is the loop header**. `continue` jumps to the condition block.

```
preheader —Jump—▶ do.body (Phis) —…—▶ do.cond —If—▶ do.body | do.exit
```

```
R IRBuilderVisitor::visitDoWhile(DoWhileStatement* stmt) {
    BasicBlock* body = createBlock("do.body");
    BasicBlock* cond = createBlock("do.cond");
    BasicBlock* exit = createBlock("do.exit");

    emitJump(body);
    currentBlock = body;
    size_t depth = scopes.size();
    auto open = openLoopHeader(body);

    jumpTargets.push_back({ exit, cond, depth, {}, {} });
    stmt->body->accept(*this);

    // do.cond joins: continue edges first, then fall-through from the body end
    vector<vector<Scope>> condIn = jumpTargets.back().continueScopes;
    if (!isTerminated()) { condIn.push_back(trimmedScopes(depth)); emitJump(cond); }
    currentBlock = cond;
```

```
    mergeScopes(cond, condIn);

    auto c = value(stmt->condition.get());
    vector<Scope> exitScopes = scopes;
    emitBranch(c, body, exit);                          // back edge: body pred #1
    closeLoopHeader(body, open, { trimmedScopes(depth) });

    JumpTarget t = std::move(jumpTargets.back()); jumpTargets.pop_back();
    currentBlock = exit;
    vector<vector<Scope>> exitIn = { exitScopes };
    for (auto& b : t.breakScopes) exitIn.push_back(b);
    mergeScopes(exit, exitIn);
    return true;
}
```

If `do.cond` has no predecessors (the body always breaks or returns), it ends up dead. The loop is then not a loop, and the reachability sweep removes `do.cond`. The `If` there still adds an edge into `do.body`, so the sweep must also drop that edge and its Phi operand (§16).

12.9 `visitFor`

AST: `init` (Statement, may be null), `test` (may be null), `update` (may be null), and `body`.

Today

1. Push a `Block` scope and visit `init`.
2. **Evaluate `test` in the pre-header, once** (IR7). With `for (;;)`, `test` is null and this crashes.
3. Create header, body and merge blocks. Push `If cond` into the **header first**, then append Phis **after** it (breaks C4).
4. Visit the body, then `update`, both in the body block. Then the same stale `If` back edge as `while`.
5. Pop the scope.

Design

```
preheader: init ; Jump header
header:    Phis ; cond ; If cond -> body, exit        (no test: Jump body)
body:      … ; Jump update
update:    [Phis merging continue edges] ; update ; Jump header
exit:      [Phis merging break edges]
```

```
R IRBuilderVisitor::visitFor(ForStatement* stmt) {
    scopes.push_back({ Scope::Type::Block, nullptr, {} });      // for (let i …) scope
    if (stmt->init) stmt->init->accept(*this);

    BasicBlock* header = createBlock("for.header");
    BasicBlock* body   = createBlock("for.body");
    BasicBlock* update = createBlock("for.update");
    BasicBlock* exit   = createBlock("for.exit");

    emitJump(header);
    currentBlock = header;
    size_t depth = scopes.size();
    auto open = openLoopHeader(header);

    vector<Scope> headerScopes;
    if (stmt->test) {
        auto cond = value(stmt->test.get());
        headerScopes = scopes;
```

```
        emitBranch(cond, body, exit);
    } else {
        emitJump(body);                              // for(;;): exit reachable only via break
    }

    jumpTargets.push_back({ exit, update, depth, {}, {} });
    currentBlock = body;
    stmt->body->accept(*this);

    vector<vector<Scope>> updIn = jumpTargets.back().continueScopes;
    if (!isTerminated()) { updIn.push_back(trimmedScopes(depth)); emitJump(update); }
    currentBlock = update;
    if (!updIn.empty()) mergeScopes(update, updIn);
    if (stmt->update) stmt->update->accept(*this);
    vector<Scope> latch = trimmedScopes(depth);
    emitJump(header);
    closeLoopHeader(header, open, { latch });

    JumpTarget t = std::move(jumpTargets.back()); jumpTargets.pop_back();
    currentBlock = exit;
    vector<vector<Scope>> exitIn;
    if (stmt->test) exitIn.push_back(headerScopes);
    for (auto& b : t.breakScopes) exitIn.push_back(b);
    if (!exitIn.empty()) mergeScopes(exit, exitIn);
    scopes.pop_back();                               // drop the for-scope (i is dead after)
    return true;
}
```

Emits (proposed) for `for (let i = 0; i < 3; i = i + 1) { print(i); }`:

```
  entry0:
    %0 = Zero
    Jump for.header1
  for.header1:  preds=[entry0, for.update3]
    %1 = Phi %0, %5          ; i
    %2 = Constant 3
    %3 = LessThan %1, %2
    If %3 -> for.body2, for.exit4
  for.body2:  preds=[for.header1]
    Print %1
    Jump for.update3
  for.update3:  preds=[for.body2]
    %4 = Constant 1
    %5 = Add %1, %4
    Jump for.header1
  for.exit4:  preds=[for.header1]
    ...
```

If an `update` block has no predecessors (every path through the body breaks or returns), the `Jump header` it emits creates an edge from a dead block. That's legal while building, and cleaned up by §16.

12.10 `visitBreak` / `visitContinue`

AST: `label` (string). There's no `LabeledStatement` node, so labels can never be targeted. Treat a non-empty label as unsupported.

Today: stubs that return `true`. `break` is silently ignored, so loops run past it.

Recommended

```
R IRBuilderVisitor::visitBreak(BreakStatement* stmt) {
    if (jumpTargets.empty()) throw std::runtime_error("SyntaxError: Illegal break statement");
    if (!stmt->label.empty()) unsupported("labelled break");
    auto& t = jumpTargets.back();
    // run pending finally blocks between here and the target (§14.6)
    t.breakScopes.push_back(trimmedScopes(t.scopeDepth));   // snapshot BEFORE the edge is
added …
    emitJump(t.breakTarget);                                 // … so the orders stay lined up
    startUnreachableBlock();
    return true;
}

R IRBuilderVisitor::visitContinue(ContinueStatement* stmt) {
    for (auto it = jumpTargets.rbegin(); it != jumpTargets.rend(); ++it) {
        if (!it->continueTarget) continue;                   // skip switch entries
        it->continueScopes.push_back(trimmedScopes(it->scopeDepth));
        emitJump(it->continueTarget);
        startUnreachableBlock();
        return true;
    }
    throw std::runtime_error("SyntaxError: Illegal continue statement");
}
```

`emitJump` does nothing if the block is already terminated. Because `startUnreachableBlock()` always runs after a terminator, `currentBlock` is never terminated when `break`/`continue` runs, so the snapshot and the edge always go together.

12.11 `visitSwitch` / `visitSwitchCase`

AST: `SwitchStatement{ discriminant, cases }`, and `SwitchCase{ test (null = default), consequent: vector<Statement> }`.

Today: stubs.

Design: evaluate the discriminant once. Build a chain of test blocks using `StrictEqual`. Bodies are laid out in source order, so fall-through is just a `Jump` to the next body. `break` jumps to the exit. `default` is reached when every test fails, wherever it appears in the source.

```
R IRBuilderVisitor::visitSwitch(SwitchStatement* stmt) {
    auto d = value(stmt->discriminant.get());
    size_t n = stmt->cases.size();
    vector<BasicBlock*> bodies(n);
    for (size_t i = 0; i < n; i++) bodies[i] = createBlock("switch.case");
    BasicBlock* exit = createBlock("switch.exit");

    int defaultIdx = -1;
    vector<vector<vector<Scope>>> bodyIn(n);          // incoming snapshots per body, edge
order
    for (size_t i = 0; i < n; i++) {
        auto* c = stmt->cases[i].get();
        if (!c->test) { defaultIdx = (int)i; continue; }
        auto t  = value(c->test.get());
        auto eq = emitValue(IROp::StrictEqual, IRType::Bool, { d, t });
        BasicBlock* next = createBlock("switch.test");
        bodyIn[i].push_back(scopes);
        emitBranch(eq, bodies[i], next);
        currentBlock = next;
    }
    // every test failed
```

```
    vector<vector<Scope>> exitIn;
    if (defaultIdx >= 0) { bodyIn[defaultIdx].push_back(scopes); emitJump(bodies[defaultIdx]);
}
    else                 { exitIn.push_back(scopes);              emitJump(exit); }

    jumpTargets.push_back({ exit, nullptr, scopes.size(), {}, {} });
    for (size_t i = 0; i < n; i++) {
        if (i > 0 && !isTerminated()) {               // fall-through from the previous body
            bodyIn[i].push_back(scopes);
            emitJump(bodies[i]);
        }
        currentBlock = bodies[i];
        mergeScopes(bodies[i], bodyIn[i]);
        scopes.push_back({ Scope::Type::Block, nullptr, {} });   // the switch body is one
block scope
        for (auto& s : stmt->cases[i]->consequent) s->accept(*this);
        scopes.pop_back();
    }
    if (!isTerminated()) { exitIn.push_back(trimmedScopes(jumpTargets.back().scopeDepth));
emitJump(exit); }

    JumpTarget t = std::move(jumpTargets.back()); jumpTargets.pop_back();
    for (auto& b : t.breakScopes) exitIn.push_back(b);
    currentBlock = exit;
    mergeScopes(exit, exitIn);
    return true;
}

R IRBuilderVisitor::visitSwitchCase(SwitchCase*) { unsupported("SwitchCase outside switch"); }
```

> **Ordering check.** All test edges into the bodies are added in the first loop, *before* any fall-through edge. So for each body, `predecessors = [test edge, fall-through edge]`, which matches the order the snapshots were pushed into `bodyIn[i]`. The edges into `exit` are added in the order *no-default edge → last body's fall-off → breaks*. The breaks are added *during* the body loop, which is **before** the last body's fall-off. To keep `exitIn` in edge order, push the break snapshots in the order their edges were created. The simplest way: have `visitBreak` push into one shared incoming list that the no-default edge has already been pushed onto. Whatever bookkeeping you choose, the rule is **one snapshot per edge, appended in the same step as the edge**.

In JS, `let` scoping in `switch` covers *all* cases, and bindings carry over through fall-through. A single block scope pushed around the whole body loop would be more exact than one per case. The simplification above is safe as long as no case reads another case's `let` binding.

12.12 `visitForIn` / `visitForOf`

AST: `ForInStatement{ init (Statement), object, body }` and `ForOfStatement{ left (Statement), right, body }`. `init/left` is a `VariableStatement` with one declarator and no initialiser, or an `ExpressionStatement` holding an identifier.

Today: `visitForOf` is a stub. `visitForIn` is half-written:

```
auto obj_reg = stmt->object->accept(*this);
scopes.push_back(Block);                          // never popped (N5)
BasicBlock* for_of_body = createBlock("for.of.body");   // orphan, EMPTY block (N5)
auto init_reg = stmt->init->accept(*this);        // declares the loop variable as an undefined
```

```
value
return true;
```

N5 (new): the leaked scope corrupts every lookup and declaration that follows. The orphan empty block breaks C1, and Turbine computes `instructions.size() - 1` on it, which underflows, as in T5, and crashes. **Any program with a `for…in` loop crashes today.** Until the real implementation lands, at minimum replace the body with `unsupported("for-in")`.

Design: both loops lower to the iterator protocol through `CallBuiltin`. `for…in` iterates an iterator over the object's enumerable keys:

```
%it   = CallBuiltin "getIterator" %obj              ; for-in: "forInIterator"
Jump header
header:  Phis
         %res  = CallBuiltin "iteratorNext" %it
         %done = LoadProperty %res "done"
         If %done -> exit, body
body:    %val  = LoadProperty %res "value"
         <bind left := %val>                         ; declare (let/const/var) or store
(identifier)
         … ; Jump header
exit:
```

```
R IRBuilderVisitor::lowerIteration(Statement* left, Expression* iterable,
                                   Statement* body, const char* iterBuiltin) {
    auto obj = value(iterable);
    IRInstruction gi(IROp::CallBuiltin, createTemp(IRType::Object), { obj });
    gi.label = iterBuiltin; emit(gi);
    auto it = gi.result;

    scopes.push_back({ Scope::Type::Block, nullptr, {} });
    BasicBlock* header = createBlock("iter.header");
    BasicBlock* bodyB  = createBlock("iter.body");
    BasicBlock* exit   = createBlock("iter.exit");
    emitJump(header);
    currentBlock = header;
    size_t depth = scopes.size();
    auto open = openLoopHeader(header);

    IRInstruction nx(IROp::CallBuiltin, createTemp(IRType::Object), { it });
    nx.label = "iteratorNext"; emit(nx);
    IRInstruction dn(IROp::LoadProperty, createTemp(IRType::Bool), { nx.result });
    dn.immediate = string("done"); emit(dn);
    vector<Scope> headerScopes = scopes;
    emitBranch(dn.result, exit, bodyB);

    jumpTargets.push_back({ exit, header, depth, {}, {} });
    currentBlock = bodyB;
    IRInstruction vl(IROp::LoadProperty, createTemp(IRType::Any), { nx.result });
    vl.immediate = string("value"); emit(vl);
    bindLoopTarget(left, vl.result);              // declare or store
    body->accept(*this);

    vector<vector<Scope>> back = jumpTargets.back().continueScopes;
    if (!isTerminated()) { back.push_back(trimmedScopes(depth)); emitJump(header); }
    closeLoopHeader(header, open, back);
    JumpTarget t = std::move(jumpTargets.back()); jumpTargets.pop_back();

    currentBlock = exit;
    vector<vector<Scope>> exitIn = { headerScopes };
    for (auto& b : t.breakScopes) exitIn.push_back(b);
```

```
    mergeScopes(exit, exitIn);
    scopes.pop_back();
    return true;
}
R IRBuilderVisitor::visitForOf(ForOfStatement* s) { return lowerIteration(s->left.get(),  s-
>right.get(),  s->body.get(), "getIterator"); }
R IRBuilderVisitor::visitForIn(ForInStatement* s) { return lowerIteration(s->init.get(),  s-
>object.get(), s->body.get(), "forInIterator"); }
```

`bindLoopTarget` looks at `left`. For a `VariableStatement`, it calls `declare(decl.id, val, kind)`. For an `ExpressionStatement` wrapping an identifier, it calls `store(name, val)`.

> **Edge order:** `emitBranch(done, exit, bodyB)` adds the `exit` edge first, so `exitIn[0]` is the header snapshot. Good.

12.13 `visitThrow`

AST: `argument`.

Today: a stub. `throw` is ignored.

```
R IRBuilderVisitor::visitThrow(ThrowStatement* stmt) {
    auto v = value(stmt->argument.get());
    IRInstruction th(IROp::Throw, nullptr, { v });
    emit(th);                    // terminator; inside try → the handler takes over at runtime
(§14.6)
    startUnreachableBlock();
    return true;
}
```

Turbine lowers it to `kThrow` (the VM must implement it).

13. Expressions

13.1 `visitBinary` and `getBinaryOp`

AST: `left`, `op` (Token), `right`. The parser uses `BinaryExpression` for **every** binary operator: arithmetic, comparison, bitwise, `&&`, `||`, `??`, and all assignment operators (§6.3).

Today

```
if op is any assignment operator → emitAssignment(expr); return true      // IR10: no value
result = createTemp(Any)                                                   // allocated BEFORE
operands
op     = getBinaryOp(token)                                                // throws on in /
instanceof
lhs    = left->accept ; rhs = right->accept                                // left-to-right:
correct
emit  result = op lhs, rhs
```

`getBinaryOp` maps tokens one-to-one: `ADD→Add`, `MINUS→Subtract`, `MUL→Multiply`, `DIV→Divide`, `MODULI→Modulo`, `POWER→Power`, `VALUE_EQUAL→Equal`, `REFERENCE_EQUAL→StrictEqual`, `INEQUALITY→NotEqual`, `STRICT_INEQUALITY→StrictNotEqual`, `LESS_THAN(_EQUAL)`, `GREATER_THAN(_EQUAL)`, `BITWISE_AND/OR/XOR`, `BITWISE_LEFT/RIGHT_SHIFT`, `UNSIGNED_RIGHT_SHIFT`, plus **`LOGICAL_AND→LogicalAnd`, `LOGICAL_OR→LogicalOr`, `NULLISH_COALESCING→NullishCoalescing`**.

Emits (today) for `return a * 2;` where `a` is captured (from `08-worked-example.md`):

```
%8 = LoadContextSlot slot=0 depth=0
%9 = Constant 2
%7 = Multiply %8, %9        ; %7 was allocated before its operands
Return %7
```

Defects

- **N3 (new):** `&&`, `||` and `??` evaluate **both** operands. In `ok && launch()`, `launch()` always runs, and `x || (x = 1)` always assigns. That's a semantic bug, separate from Turbine dropping the ops (T2). They must lower to control flow (§13.2).
- `in` and `instanceof` throw "Unknown binary operator".
- The result temp is numbered before its operands. That's cosmetic, but dumps read backwards. Allocate the result **after** evaluating the operands.
- Every result is typed `Any`. Use the type rules from §3.2.

Recommended implementation

```
R IRBuilderVisitor::visitBinary(BinaryExpression* expr) {
    switch (expr->op.type) {
        case TokenType::ASSIGN:       case TokenType::ASSIGN_ADD:   case
TokenType::ASSIGN_MINUS:
        case TokenType::ASSIGN_MUL:   case TokenType::ASSIGN_DIV:   case
TokenType::MODULI_ASSIGN:
        case TokenType::POWER_ASSIGN: case TokenType::BITWISE_LEFT_SHIFT_ASSIGN:
        case TokenType::BITWISE_RIGHT_SHIFT_ASSIGN: case
TokenType::UNSIGNED_RIGHT_SHIFT_ASSIGN:
        case TokenType::BITWISE_AND_ASSIGN: case TokenType::BITWISE_OR_ASSIGN:
        case TokenType::BITWISE_XOR_ASSIGN: case TokenType::LOGICAL_AND_ASSIGN:
        case TokenType::LOGICAL_OR_ASSIGN:  case TokenType::NULLISH_COALESCING_ASSIGN:
            return emitAssignment(expr->left.get(), expr->op, expr->right.get());   // §13.3

        case TokenType::LOGICAL_AND: case TokenType::LOGICAL_OR: case
TokenType::NULLISH_COALESCING:
            return emitShortCircuit(expr->op.type, expr->left.get(), expr->right.get());  //
§13.2
        default: break;
    }

    auto lhs = value(expr->left.get());
    auto rhs = value(expr->right.get());

    // `in` / `instanceof` (token names as in the commented-out lines of getBinaryOp)
    // → CallBuiltin "in" / "instanceof" with { lhs, rhs }, result type Bool.

    IROp op = getBinaryOp(expr->op);
    return emitValue(op, binaryResultType(op, lhs, rhs), { lhs, rhs });
}

IRType binaryResultType(IROp op, const shared_ptr<IRValue>& a, const shared_ptr<IRValue>& b) {
    switch (op) {
```

```
        case IROp::Equal: case IROp::NotEqual: case IROp::StrictEqual: case
IROp::StrictNotEqual:
        case IROp::LessThan: case IROp::LessThanOrEqual: case IROp::GreaterThan:
        case IROp::GreaterThanOrEqual:
            return IRType::Bool;
        case IROp::Add:
            if (a->type == IRType::String || b->type == IRType::String) return IRType::String;
            if (a->type == IRType::Number && b->type == IRType::Number) return IRType::Number;
            return IRType::Any;
        default:
            return IRType::Number;      // - * / % ** and all bitwise ops always produce
numbers
    }
}
```

Building-time constant folding? Leave it out of the builder. Folding belongs in Pharoah, which sees the whole function after SSA construction. Folding in the builder only catches literal-against-literal cases and mixes two concerns. The one exception worth making is negative numeric literals (§13.4).

13.2 Short-circuit operators: `&&`, `||`, `??` (and `visitLogical`)

Semantics

Expression	Value
`a && b`	`ToBoolean(a) ? b : a`
`a || b`	`ToBoolean(a) ? a : b`
`a ?? b`	`(a === null || a === undefined) ? b : a`

The right operand must run **only** on the path that needs it. So this is control flow, plus a Phi that picks the result.

Recommended implementation

```
shared_ptr<IRValue> IRBuilderVisitor::emitShortCircuit(TokenType op, Expression* l, Expression*
r) {
    auto lhs = value(l);
    shared_ptr<IRValue> test = lhs;
    if (op == TokenType::NULLISH_COALESCING) {
        auto nul = emitValue(IROp::Null, IRType::Null);
        test = emitValue(IROp::Equal, IRType::Bool, { lhs, nul });     // loose == null ⇔ null
or undefined
    }

    BasicBlock* rhsB = createBlock("sc.rhs");
    BasicBlock* endB = createBlock("sc.end");

    vector<vector<Scope>> in = { scopes };            // edge #0 into endB: the short-circuit
edge
    vector<shared_ptr<IRValue>> vals = { lhs };
    if (op == TokenType::LOGICAL_OR) emitBranch(test, endB, rhsB);   // truthy → done
    else                             emitBranch(test, rhsB, endB);   // &&: truthy → eval rhs;
??: nullish → eval rhs

    currentBlock = rhsB;
    auto rhs = value(r);                               // may contain assignments or nested
control flow
    in.push_back(scopes); vals.push_back(rhs);
```

```
    emitJump(endB);

    currentBlock = endB;
    mergeScopes(endB, in);                              // Phis for variables assigned in rhs
    auto dst = createTemp(vals[0]->type == vals[1]->type ? vals[0]->type : IRType::Any);
    IRInstruction phi(IROp::Phi, dst, vals);
    phi.label = "<logical>";
    endB->instructions.push_back(phi);                  // still in the Phi prefix: nothing else
emitted yet
    return dst;
}

R IRBuilderVisitor::visitLogical(LogicalExpression* e) {           // not built by today's
parser
    return emitShortCircuit(e->op.type, e->left.get(), e->right.get());
}
```

`emitBranch` adds the `endB` edge first for `||` and second for `&&`/`??`. In both cases it's the **first edge into `endB`**, because the `rhs → end` edge comes later. So `in[0]` and `vals[0]` always belong to `predecessors[0]`.

Emits (proposed) for `let r = a && f();`:

```
  entry0:
    ...                       ; %a = value of a
    If %a -> sc.rhs1, sc.end2
  sc.rhs1:  preds=[entry0]
    %c = Call %f
    Jump sc.end2
  sc.end2:  preds=[entry0, sc.rhs1]
    %r = Phi %a, %c           ; <logical>
```

When an `&&` is used directly as an `if` or loop condition, materialising the Phi is wasteful, since control could branch straight to the targets. A later refinement, `emitCondition(expr, ifTrue, ifFalse)`, does this: it compiles `&&`/`||`/`!` as jumps. That's the classic "jumping code" scheme. Get the value-producing version right first.

13.3 Assignment: `emitAssignment` and `visitAssignment`

Today

```
void emitAssignment(BinaryExpression* expr) {
    value = expr->right->accept           // RHS evaluated FIRST
    if (op == ASSIGN && left is Identifier) store(name, value);
    // every other operator, and every non-identifier target: silently ignored
}
```

Defects: no value is returned (IR10). Compound operators are dropped (IR11). Member targets (`o.p = v`, `a[i] = v`) are dropped silently. The evaluation order is wrong for members: JS evaluates `o` and `i` **before** the right-hand side.

Design: references. A target is resolved once into a *reference*. The reference can then be read (for compound operators) and written:

```
struct Reference {
    enum class Kind { Variable, Named, Keyed } kind;
```

```
    string name;                    // Variable: variable name; Named: property name
    shared_ptr<IRValue> object;     // Named / Keyed
    shared_ptr<IRValue> key;        // Keyed
};

Reference IRBuilderVisitor::evaluateReference(Expression* target) {
    if (auto* id = dynamic_cast<IdentifierExpression*>(target))
        return { Reference::Kind::Variable, id->name, nullptr, nullptr };
    if (auto* m = dynamic_cast<MemberExpression*>(target)) {
        auto obj = value(m->object.get());
        if (!m->computed) return { Reference::Kind::Named, m->name.lexeme, obj, nullptr };
        return { Reference::Kind::Keyed, "", obj, value(m->property.get()) };
    }
    throw std::runtime_error("SyntaxError: Invalid left-hand side in assignment");
}

shared_ptr<IRValue> IRBuilderVisitor::loadReference(const Reference& r) {
    switch (r.kind) {
        case Reference::Kind::Variable: return lookup(r.name);
        case Reference::Kind::Named: {
            IRInstruction ld(IROp::LoadProperty, createTemp(IRType::Any), { r.object });
            ld.immediate = r.name; emit(ld); return ld.result;
        }
        case Reference::Kind::Keyed:
            return emitValue(IROp::LoadElement, IRType::Any, { r.object, r.key });
    }
}

void IRBuilderVisitor::storeReference(const Reference& r, shared_ptr<IRValue> v) {
    switch (r.kind) {
        case Reference::Kind::Variable: store(r.name, v); return;
        case Reference::Kind::Named: {
            IRInstruction st(IROp::StoreProperty, nullptr, { r.object, v });
            st.immediate = r.name; emit(st); return;
        }
        case Reference::Kind::Keyed: {
            IRInstruction st(IROp::StoreElement, nullptr, { r.object, r.key, v });
            emit(st); return;
        }
    }
}
```

Compound operator table

Token	Op	Token	Op
`ASSIGN_ADD`	`Add`	`BITWISE_LEFT_SHIFT_ASSIGN`	`ShiftLeft`
`ASSIGN_MINUS`	`Subtract`	`BITWISE_RIGHT_SHIFT_ASSIGN`	`ShiftRight`
`ASSIGN_MUL`	`Multiply`	`UNSIGNED_RIGHT_SHIFT_ASSIGN`	`UnsignedShiftRight`
`ASSIGN_DIV`	`Divide`	`BITWISE_AND_ASSIGN`	`BitAnd`
`MODULI_ASSIGN`	`Modulo`	`BITWISE_OR_ASSIGN`	`BitOr`
`POWER_ASSIGN`	`Power`	`BITWISE_XOR_ASSIGN`	`BitXor`

Recommended implementation

```
shared_ptr<IRValue> IRBuilderVisitor::emitAssignment(Expression* target, const Token& op,
Expression* rhsExpr) {
    switch (op.type) {
        case TokenType::LOGICAL_AND_ASSIGN:
```

```
        case TokenType::LOGICAL_OR_ASSIGN:
        case TokenType::NULLISH_COALESCING_ASSIGN:
            return emitLogicalAssignment(target, op.type, rhsExpr);
        default: break;
    }
    Reference ref = evaluateReference(target);        // object/key first: JS order
    shared_ptr<IRValue> v;
    if (op.type == TokenType::ASSIGN) {
        v = value(rhsExpr);
    } else {
        auto cur = loadReference(ref);
        auto rhs = value(rhsExpr);
        IROp bop = compoundOp(op.type);
        v = emitValue(bop, binaryResultType(bop, cur, rhs), { cur, rhs });
    }
    storeReference(ref, v);
    return v;                                          // `a = b = 3` and `if ((x = f()))` now
work
}

R IRBuilderVisitor::visitAssignment(AssignmentExpression* e) {    // not built by today's
parser
    return emitAssignment(e->left.get(), e->op, e->right.get());
}
```

`x &&= v` means `x && (x = v)`: the store happens **only** on the path that evaluates `v`. `emitLogicalAssignment` is `emitShortCircuit` with the right-hand block doing `storeReference(ref, value(rhs))`. The reference is evaluated once, before the branch.

Emits (proposed)

```
let x = 1; x += 2;         // SSA variable
```

```
%0 = Constant 1
%1 = Constant 2
%2 = Add %0, %1            ; x → %2, no store instruction
```

```
o.count += 1;              // property
```

```
%o = ...
%3 = LoadProperty %o "count"
%4 = Constant 1
%5 = Add %3, %4
StoreProperty %o, %5 "count"
```

13.4 `visitUnary`

AST: `op` (Token) and `right`. The parser creates it for `! ~ ++ -- + - typeof delete void`. `await` becomes an `AwaitExpression` instead. **Prefix `++x` / `--x` arrive here, not in `visitUpdate`.**

Today: a stub that returns `true`. `let y = -3;` aborts with `bad_variant_access` (IR3).

Lowering table

Operator	IR
`-x`	`Neg %x`. If `x` is a numeric literal, fold to a negative `Constant`/`HeapNumber` (`-0` → `HeapNumber -0.0`).
`+x`	`ToNumber %x`
`!x`	`Not %x` → Bool
`~x`	`BitXor %x, Constant -1` (no new opcode needed)
`void x`	evaluate `x`, then `Undefined`
`typeof x`	`TypeOf %x` (new op). An undeclared identifier must **not** throw: it yields `"undefined"`.
`delete o.p` / `delete o[k]`	`DeleteProperty %o, key` (new op). For non-member operands: evaluate, then `True`.
`++x` / `--x`	`emitUpdate(x, inc, prefix=true)` (§13.5)

```
R IRBuilderVisitor::visitUnary(UnaryExpression* expr) {
    switch (expr->op.type) {
        case TokenType::INCREMENT: return emitUpdate(expr->right.get(), true,
/*prefix=*/true);
        case TokenType::DECREMENT: return emitUpdate(expr->right.get(), false,
/*prefix=*/true);
        case TokenType::DELETE:    return emitDelete(expr->right.get());
        case TokenType::MINUS:
            if (auto* lit = dynamic_cast<NumericLiteral*>(expr->right.get()))
                return emitNumber(-toValue(lit->value).numberValue);   // same
Zero/Constant/HeapNumber logic as §11.3
            break;
        case TokenType::TYPEOF:
            if (auto* id = dynamic_cast<IdentifierExpression*>(expr->right.get()); id &&
!isResolvable(id->name)) {
                IRInstruction s(IROp::StringConstant, createTemp(IRType::String), {});
                s.immediate = string("undefined"); emit(s); return s.result;
            }
            break;
        default: break;
    }
    auto v = value(expr->right.get());
    switch (expr->op.type) {
        case TokenType::MINUS:       return emitValue(IROp::Neg,      IRType::Number, { v });
        case TokenType::ADD:         return emitValue(IROp::ToNumber, IRType::Number, { v });
        case TokenType::LOGICAL_NOT: return emitValue(IROp::Not,      IRType::Bool,   { v });
        case TokenType::BITWISE_NOT: return emitValue(IROp::BitXor,   IRType::Number, { v,
emitNumber(-1) });
        case TokenType::VOID:        return emitUndefined();
        case TokenType::TYPEOF:      return emitValue(IROp::TypeOf,   IRType::String, { v });
        default: unsupported("unary operator");
    }
}
```

13.5 `visitUpdate` (postfix `x++`, `x--`)

AST: `op`, `argument`, `prefix` (always `false` from today's parser).

Today: a stub. `i++` does nothing, so `for (…; …; i++)` loops forever once loops work.

Semantics: `x++` reads `x`, converts it to a number `n`, stores `n + 1`, and evaluates to `n`. Note: `n`, not the original value. `"5"++` evaluates to `5`.

```
shared_ptr<IRValue> IRBuilderVisitor::emitUpdate(Expression* target, bool inc, bool prefix) {
    Reference ref = evaluateReference(target);
    auto oldV = loadReference(ref);
    auto num  = emitValue(IROp::ToNumber, IRType::Number, { oldV });
    auto one  = emitNumber(1);
    auto newV = emitValue(inc ? IROp::Add : IROp::Subtract, IRType::Number, { num, one });
    storeReference(ref, newV);
    return prefix ? newV : num;
}

R IRBuilderVisitor::visitUpdate(UpdateExpression* e) {
    return emitUpdate(e->argument.get(), e->op.type == TokenType::INCREMENT, e->prefix);
}
```

Emits (proposed) for `i++` with `i` as an SSA variable `%1`:

```
%6 = ToNumber %1
%7 = Constant 1
%8 = Add %6, %7            ; i → %8; the expression's value is %6
```

If `i` is known to be a `Number`, Pharoah removes the `ToNumber`.

13.6 `visitConditional` (`c ? a : b`)

AST: `test`, `consequent`, `alternate`.

Today: a stub.

Design: the same diamond as `if`, plus a Phi for the value.

```
R IRBuilderVisitor::visitConditional(ConditionalExpression* e) {
    auto cond = value(e->test.get());
    BasicBlock* tB = createBlock("cond.then");
    BasicBlock* fB = createBlock("cond.else");
    BasicBlock* endB = createBlock("cond.end");
    vector<Scope> before = scopes;
    emitBranch(cond, tB, fB);

    vector<vector<Scope>> in; vector<shared_ptr<IRValue>> vals;
    currentBlock = tB; scopes = before;
    vals.push_back(value(e->consequent.get())); in.push_back(scopes); emitJump(endB);
    currentBlock = fB; scopes = before;
    vals.push_back(value(e->alternate.get()));  in.push_back(scopes); emitJump(endB);

    currentBlock = endB;
    mergeScopes(endB, in);
    auto dst = createTemp(vals[0]->type == vals[1]->type ? vals[0]->type : IRType::Any);
    IRInstruction phi(IROp::Phi, dst, vals); phi.label = "<cond>";
    endB->instructions.push_back(phi);
    return dst;
}
```

13.7 `visitSequence` and `visitComma`

AST: `SequenceExpression{ expressions }` and `CommaExpression{ left, right }`.

Today: stubs.

```
R IRBuilderVisitor::visitSequence(SequenceExpression* e) {
    shared_ptr<IRValue> last;
    for (auto& x : e->expressions) last = value(x.get());
    return last ? last : emitUndefined();
}
R IRBuilderVisitor::visitComma(CommaExpression* e) {
    value(e->left.get());
    return value(e->right.get());
}
```

13.8 `visitCall`

AST: `callee` and `arguments`.

Today

- `print(...)` is intercepted when the callee is the identifier `print`: emit `Print args…` and **return `true`**.
- Otherwise: evaluate the callee, allocate the result temp, evaluate the arguments left to right, then emit `Call callee, args…`.

Emits (today) for `print(fn1());`:

```
%14 = Call %12
Print %14
```

Defects

- **N1 (new):** `print` produces no value, so `let r = print(1);` and `f(print(1))` abort with `bad_variant_access`.
- A user-defined function named `print` can never be called. The intercept should apply only when `print` doesn't resolve to a binding.
- Method calls (`o.m(x)`) can't work: `visitMember` returns `true`, and even with a working member load, the receiver `o` would be lost.
- Spread arguments (`f(...xs)`) reach `visitSpreadExpression`, which returns `true`, and abort.
- Arguments arrive at DeathStar but are thrown away (V4). That's a VM issue, but it's the reason parameters can't be tested end to end.

Recommended implementation

```
R IRBuilderVisitor::visitCall(CallExpression* expr) {
    auto* calleeId = dynamic_cast<IdentifierExpression*>(expr->callee.get());
    if (calleeId && calleeId->name == "print" && !isResolvable("print")) {
        vector<shared_ptr<IRValue>> args;
        for (auto& a : expr->arguments) args.push_back(value(a.get()));
        emit(IRInstruction(IROp::Print, nullptr, args));
        return emitUndefined();                                  // N1
    }

    bool hasSpread = std::any_of(expr->arguments.begin(), expr->arguments.end(),
        [](auto& a){ return dynamic_cast<SpreadExpression*>(a.get()) != nullptr; });

    shared_ptr<IRValue> receiver, fn;
    if (auto* m = dynamic_cast<MemberExpression*>(expr->callee.get())) {
        receiver = value(m->object.get());                       // evaluate o ONCE
```

```
        fn = m->computed ? emitValue(IROp::LoadElement, IRType::Any, { receiver, value(m-
>property.get()) })
                         : emitLoadProperty(receiver, m->name.lexeme);
    } else {
        fn = value(expr->callee.get());
    }

    if (hasSpread) {
        auto argv = buildArrayFrom(expr->arguments);                    // §13.11
        IRInstruction c(IROp::CallBuiltin, createTemp(IRType::Any),
                        { fn, receiver ? receiver : emitUndefined(), argv });
        c.label = "callWithSpread"; emit(c); return c.result;
    }

    vector<shared_ptr<IRValue>> ops = { fn };
    if (receiver) ops.push_back(receiver);
    for (auto& a : expr->arguments) ops.push_back(value(a.get()));
    return emitValue(receiver ? IROp::CallProperty : IROp::Call, IRType::Any, ops);
}
```

Emits (proposed) for `list.push(x + 1)`:

```
%10 = ... list
%11 = LoadProperty %10 "push"
%12 = ... x
%13 = Constant 1
%14 = Add %12, %13
%15 = CallProperty %11, %10, %14      ; callee, receiver, args…
```

13.9 `visitMember`

AST: `object`, `computed`. When `computed` is false, the property name is in `name` (a Token) and `property` is null. When it's true, the key expression is in `property`.

Today: a stub.

```
shared_ptr<IRValue> IRBuilderVisitor::emitLoadProperty(shared_ptr<IRValue> obj, const string&
name) {
    IRInstruction ld(IROp::LoadProperty, createTemp(IRType::Any), { obj });
    ld.immediate = name;
    emit(ld);
    return ld.result;
}

R IRBuilderVisitor::visitMember(MemberExpression* e) {
    auto obj = value(e->object.get());
    if (!e->computed) return emitLoadProperty(obj, e->name.lexeme);
    auto key = value(e->property.get());
    return emitValue(IROp::LoadElement, IRType::Any, { obj, key });
}
```

`a.length`, where `a` is known to be an `Array`, can later get a dedicated op. For now `LoadProperty "length"` is correct. Lowering: `kLdaNamedProperty obj, constIdx(name)` and `kLdaKeyedProperty obj` (with the key in the accumulator).

13.10 `visitNew`

AST: `callee` and `arguments`.

Today: a stub.

```
R IRBuilderVisitor::visitNew(NewExpression* e) {
    vector<shared_ptr<IRValue>> ops = { value(e->callee.get()) };
    for (auto& a : e->arguments) ops.push_back(value(a.get()));   // spread: same treatment as
calls
    return emitValue(IROp::Construct, IRType::Object, ops);
}
```

At runtime, `Construct` allocates an object with `prototype = callee.prototype`, calls the callee with that object as `this`, and returns the object, unless the callee returned an object of its own.

13.11 `visitArray` and `visitSpreadExpression`

AST: `ArrayLiteralExpression{ elements }`. An element may be a `SpreadExpression{ expression }`.

Today: both are stubs.

Design: `NewArray` with a length hint, then `StoreElement` at constant indices. Once a spread appears, later indices aren't known at compile time, so switch to `CallBuiltin "arrayPush"` / `"arrayPushSpread"`.

```
shared_ptr<IRValue> IRBuilderVisitor::buildArrayFrom(const vector<unique_ptr<Expression>>&
elems) {
    IRInstruction na(IROp::NewArray, createTemp(IRType::Array), {});
    na.immediate = static_cast<int>(elems.size());
    emit(na);
    auto arr = na.result;
    bool dynamicIndex = false;
    for (size_t i = 0; i < elems.size(); i++) {
        if (auto* sp = dynamic_cast<SpreadExpression*>(elems[i].get())) {
            IRInstruction p(IROp::CallBuiltin, nullptr, { arr, value(sp->expression.get()) });
            p.label = "arrayPushSpread"; emit(p);
            dynamicIndex = true;
            continue;
        }
        auto v = value(elems[i].get());
        if (dynamicIndex) {
            IRInstruction p(IROp::CallBuiltin, nullptr, { arr, v });
            p.label = "arrayPush"; emit(p);
        } else {
            emit(IRInstruction(IROp::StoreElement, nullptr, { arr, emitNumber((double)i), v
}));
        }
    }
    return arr;
}

R IRBuilderVisitor::visitArray(ArrayLiteralExpression* e) { return buildArrayFrom(e->elements);
}

R IRBuilderVisitor::visitSpreadExpression(SpreadExpression*) {
    unsupported("spread outside an array literal, call, or new expression");
}
```

13.12 `visitObject`

AST: `props: vector<pair<Token, unique_ptr<Expression>>>`, where the key is a token: an identifier, a string or a number.

Today: a stub.

```
R IRBuilderVisitor::visitObject(ObjectLiteralExpression* e) {
    auto obj = emitValue(IROp::NewObject, IRType::Object);
    for (auto& [keyTok, valExpr] : e->props) {
        auto v = value(valExpr.get());
        IRInstruction st(IROp::StoreProperty, nullptr, { obj, v });
        st.immediate = propertyKeyName(keyTok);   // strip quotes from string-literal keys if
the scanner keeps them
        emit(st);
    }
    return obj;
}
```

Shorthand properties (`{ a }`) and methods (`{ m() {} }`) depend on how the parser encodes them. If the value expression is an `IdentifierExpression` with the same name as the key, the generic path already works. Methods are just `FunctionExpression` values, built through §14.2.

13.13 `visitTemplateLiteral`

AST: `quasis` (StringLiterals) and `expressions`, interleaved as `q0 e0 q1 e1 … qn`. The interpreter appends `quasis[i]`, then `expressions[i]` if there is one.

Today: a stub.

```
R IRBuilderVisitor::visitTemplateLiteral(TemplateLiteral* t) {
    shared_ptr<IRValue> acc;
    auto append = [&](shared_ptr<IRValue> s) {
        acc = acc ? emitValue(IROp::Add, IRType::String, { acc, s }) : s;
    };
    for (size_t i = 0; i < t->quasis.size(); i++) {
        if (!t->quasis[i]->text.empty()) append(value(t->quasis[i].get()));
        if (i < t->expressions.size())
            append(emitValue(IROp::ToString, IRType::String, { value(t->expressions[i].get())
}));
    }
    if (!acc) {                                        // `` → ""
        IRInstruction s(IROp::StringConstant, createTemp(IRType::String), {});
        s.immediate = string(""); emit(s); acc = s.result;
    }
    return acc;
}
```

The explicit `ToString` matters. `Add` with a non-string left operand would do numeric addition, so `${1}${2}` must give `"12"`, not `3`.

14. Functions, classes and advanced constructs

14.1 `visitFunction` (`FunctionDeclaration`)

AST: `id`, `params` (Expressions: `IdentifierExpression`, `BinaryExpression(ASSIGN)` for defaults, or `RestParameter`), `body` (a Statement, in practice a `BlockStatement`), and `is_async`.

Today, step by step

1. Create the child `IRFunction` named `id`, and remember `childFnPtr`.
2. Save `currentFunction`, `currentBlock`, `scopes` (moved), and `currentFunctionOwnsTopContextFrame`.
3. Switch to the child. Create its first block (named after the function). Reset `scopes` to a single `Function` scope.
4. Capture analysis (§9.2): `captured = declaredNames ∩ freeVarsOfDirectChildren`.
5. If any are captured: emit `%ctx = CreateContext` with `contextSlot = |captured|`, push a `Context` with the slot map, and set `owns = true`. Otherwise set `owns = false`.
6. Visit each statement of the body block. A non-block body is silently ignored.
7. Pop the context frame if one was pushed, pop the scope, and append the child to `irModule.functions` (post-order).
8. Restore the saved state.
9. In the **outer** function: emit `%f = Closure`, with `childFunction = child` and the operand `contexts.back().contextValue` if any context is active.
10. `declare(id, %f, Var)`.

Emits (today) for the closure in the worked example:

```
function #0 'u'                          function #1 'n'
  u2:                                      n1:
    %4 = Constant 7                          %1 = CreateContext slot=1
    StoreContextSlot %4 slot=0 depth=0       %2 = Constant 90
    ...                                      StoreCurrentContextSlot %2 slot=0
    Return %7                                ...
                                             %10 = Closure u, %1
                                             Return %10
```

Defects

	Defect	Fix
IR2	parameters are never declared	`Parameter` + `bindParameter` (below)
IR8	capture analysis is shallow	resolver pre-pass (§9.3)
IR9	no implicit `return undefined`	emit one if the last block isn't terminated
N4	the closure operand may be another function's register	capture the frame's current context (§10.3)
N6	top-level bindings are invisible to functions	treat the top level as a function (§11.1)
N7	a function can't call itself (`function f(){ f(); }`): its name is declared only *after* its body is built, and only in the outer function's SSA scope	hoist the name (below); the resolver context-allocates it if the body references it

	Defect	Fix
N8	`jumpTargets` (once added) must be saved and cleared: a `break` inside a nested function must not target the outer loop	save/restore in `buildFunction`
–	the non-block body is dropped	assert or handle
–	`is_async` is ignored	§14.9

Recommended design: one `buildFunction` for every function form

```
struct FunctionShape {
    const void* key;                    // AST node: lookup key into resolver.info
    string name;
    vector<Expression*> params;         // flattened
    Statement* body = nullptr;          // block body
    Expression* exprBody = nullptr;     // arrow concise body
    bool isArrow = false;
};

shared_ptr<IRValue> IRBuilderVisitor::buildFunction(const FunctionShape& f) {
    auto fn = make_unique<IRFunction>();
    IRFunction* child = fn.get();
    fn->name = f.name;

    // ---- save the outer builder state
    auto savedFn = currentFunction;  auto savedBlock = currentBlock;
    auto savedScopes = std::move(scopes);
    auto savedTargets = std::move(jumpTargets);
    bool savedOwns = currentFunctionOwnsTopContextFrame;

    currentFunction = child;
    currentBlock = createBlock(f.name.empty() ? "anon" : f.name);
    child->entry = currentBlock;
    scopes.clear(); jumpTargets.clear();
    scopes.push_back({ Scope::Type::Function, nullptr, {} });

    // ---- 1. own context first, so parameters can be stored into it
    const FunctionInfo& info = resolver.info.at(f.key);
    bool pushed = openFunctionContext(info);            // CreateContext + contexts.push_back +
owns flag

    // ---- 2. receiver and parameters (declare() routes captured ones into slots)
    if (!f.isArrow) declare("this", emitParameter(0, IRType::Any), BindingKind::Let);
    for (size_t i = 0; i < f.params.size(); i++) bindParameter(f.params[i], (int)i + 1);

    // ---- 3. body
    if (f.exprBody) {
        IRInstruction r(IROp::Return, nullptr, { value(f.exprBody) });
        emit(r);
    } else if (auto* blk = dynamic_cast<BlockStatement*>(f.body)) {
        hoistFunctionDeclarations(blk->body);
        for (auto& s : blk->body)
            if (!dynamic_cast<FunctionDeclaration*>(s.get())) s->accept(*this);
    } else {
        unsupported("function body that is not a block");
    }

    // ---- 4. implicit return (C9)
    if (!isTerminated()) {
        IRInstruction r(IROp::Return, nullptr, { emitUndefined() });
        emit(r);
    }
    if (pushed) contexts.pop_back();
```

```
    removeUnreachableBlocks(*child);                    // §16
    irModule.functions.emplace_back(std::move(fn));

    // ---- restore
    currentFunction = savedFn; currentBlock = savedBlock;
    scopes = std::move(savedScopes); jumpTargets = std::move(savedTargets);
    currentFunctionOwnsTopContextFrame = savedOwns;

    // ---- 5. the closure value, in the OUTER function (captures the frame's current context,
§10.3)
    IRInstruction cl(IROp::Closure, createTemp(IRType::Function), {});
    cl.childFunction = child;
    emit(cl);
    return cl.result;
}

R IRBuilderVisitor::visitFunction(FunctionDeclaration* stmt) {
    // Reached only if hoisting did not already handle this declaration.
    auto f = buildFunction({ stmt, stmt->id, rawPointers(stmt->params), stmt->body.get() });
    declare(stmt->id, f, BindingKind::Var);
    return true;
}

void IRBuilderVisitor::hoistFunctionDeclarations(const vector<unique_ptr<Statement>>& body) {
    for (auto& s : body)
        if (auto* fd = dynamic_cast<FunctionDeclaration*>(s.get())) {
            auto f = buildFunction({ fd, fd->id, rawPointers(fd->params), fd->body.get() });
            declare(fd->id, f, BindingKind::Var);
        }
}
```

Parameters

```
shared_ptr<IRValue> IRBuilderVisitor::emitParameter(int index, IRType t) {
    IRInstruction p(IROp::Parameter, createTemp(t), {});
    p.immediate = index;                        // 0 = receiver, 1.. = declared parameters
    emit(p);
    return p.result;
}

void IRBuilderVisitor::bindParameter(Expression* param, int index) {
    if (auto* id = dynamic_cast<IdentifierExpression*>(param)) {
        declare(id->name, emitParameter(index, IRType::Any), BindingKind::Let);
        return;
    }
    if (auto* rest = dynamic_cast<RestParameter*>(param)) {          // ...args
        IRInstruction r(IROp::CallBuiltin, createTemp(IRType::Array), {});
        r.label = "restArguments"; r.immediate = index;              // collect args[index-1
..]
        emit(r);
        declare(rest->token.lexeme, r.result, BindingKind::Let);
        return;
    }
    if (auto* def = dynamic_cast<BinaryExpression*>(param); def && def->op.type ==
TokenType::ASSIGN) {
        auto* id = dynamic_cast<IdentifierExpression*>(def->left.get());
        if (!id) unsupported("destructuring parameter");
        auto p   = emitParameter(index, IRType::Any);
        auto und = emitUndefined();
        auto isU = emitValue(IROp::StrictEqual, IRType::Bool, { p, und });
        // v = isU ? <default> : p, built like visitConditional, with the default evaluated
only when needed
        auto v = emitSelect(isU, [&]{ return value(def->right.get()); }, [&]{ return p; });
```

```
        declare(id->name, v, BindingKind::Let);
        return;
    }
    unsupported("parameter form");
}
```

Cross-layer requirement. `Parameter i` needs runtime support. Turbine lowers it to a new `kLdaArgument i`, or the convention can be fixed so that the VM copies the receiver and arguments into registers `0..n` and Turbine pre-assigns those registers to the `Parameter` values. Either way, DeathStar's `call()` must stop discarding `args` (V4).

Emits (proposed) for the counter closure:

```
function counter(start) {
    let n = start;
    return function () { n = n + 1; return n; };
}
```

```
function #0 'anon'
  anon1:
    %4 = Parameter 0                        ; this (unused; DCE removes it)
    %5 = LoadContextSlot slot=0 depth=0     ; n  (anon owns no context → depth counts from
parent)
    %6 = Constant 1
    %7 = Add %5, %6
    StoreContextSlot %7 slot=0 depth=0
    %8 = LoadContextSlot slot=0 depth=0
    Return %8
function #1 'counter'
  counter0:
    %0 = CreateContext slot=1               ; slots: n → 0
    %1 = Parameter 0                        ; this
    %2 = Parameter 1                        ; start (not captured → stays SSA)
    StoreCurrentContextSlot %2 slot=0       ; let n = start   (n is captured)
    %9 = Closure anon                       ; captures counter's own context implicitly
    Return %9
```

14.2 `visitFunctionExpression`

AST: `name` (default `"<anon>"`), `params`, `body`, `is_async`, `token`.

Today: a stub. `let f = function(){}` aborts.

```
R IRBuilderVisitor::visitFunctionExpression(FunctionExpression* e) {
    return buildFunction({ e, e->name, rawPointers(e->params), e->body.get() });
}
```

A **named** function expression (`let f = function g(){ g(); }`) binds `g` only inside its own body. The simplest correct lowering: in the outer function, open a block scope, build the closure, `declare("g", closure, Let)`, and close the scope. The resolver must treat `g` as declared in that synthetic scope, so that the body's reference to `g` gets context-allocated.

14.3 `visitArrowFunction` and `visitThis`

AST: `ArrowFunction{ parameters (one Expression: an identifier, or a parenthesised list), exprBody | stmtBody, token, is_async }`.

Today: both are stubs.

```
vector<Expression*> flattenArrowParams(Expression* p) {
    vector<Expression*> out;
    if (!p) return out;
    if (auto* s = dynamic_cast<SequenceExpression*>(p)) { for (auto& x : s->expressions)
out.push_back(x.get()); }
    else if (auto* c = dynamic_cast<CommaExpression*>(p)) {
        auto l = flattenArrowParams(c->left.get()); out.insert(out.end(), l.begin(), l.end());
        out.push_back(c->right.get());
    }
    else out.push_back(p);
    return out;
}

R IRBuilderVisitor::visitArrowFunction(ArrowFunction* a) {
    FunctionShape f{ a, a->name, flattenArrowParams(a->parameters.get()),
                     a->stmtBody.get(), a->exprBody.get(), /*isArrow=*/true };
    return buildFunction(f);
}

R IRBuilderVisitor::visitThis(ThisExpression*) { return lookup("this"); }
```

Lexical `this` for free. A normal function declares `"this"` as `Parameter 0`. An arrow doesn't declare it, so `lookup("this")` inside an arrow falls through to the context chain. Because the resolver (§9.3) recorded `this` as captured by the arrow, the enclosing function stored its `this` into a context slot. Arrows need no special runtime machinery: `this` is just another captured variable. The top level should `declare("this", emitUndefined(), Let)`, which gives module semantics.

14.4 `visitRestParameter`

`RestParameter` is only valid in a parameter list, where `bindParameter` (§14.1) consumes it. A direct visit is a builder bug: call `unsupported("rest parameter outside a parameter list")`.

14.5 Classes: `visitClass`, `visitClassExpression`, `visitMethodDefinition`, `visitSuper`, modifier keywords

AST: `ClassDeclaration{ id, superClass, body: vector<MethodDefinition>, fields: vector<PropertyDeclaration{ modifiers, property }> }` and `MethodDefinition{ name, params, methodBody, modifiers }`. The modifiers are `PublicKeyword`, `PrivateKeyword`, `ProtectedKeyword` and `StaticKeyword` expressions.

Today: all stubs.

Design: desugar to functions, objects and property stores. The IR itself needs no class concept.

```
%super = <value of superClass>                    ; if extends
%ctor  = Closure A.constructor                    ; the 'constructor' method, or a synthesised
one
```

```
%proto = LoadProperty %ctor "prototype"
CallBuiltin "inherit" %ctor, %super               ; if extends: link ctor.__proto__ and
prototype chain
for each non-static method m:
    %m = Closure A.m ; StoreProperty %proto, %m "m"
for each static method s:
    %s = Closure A.s ; StoreProperty %ctor, %s "s"
for each static field f = init:
    %v = <init> ; StoreProperty %ctor, %v "f"
declare A := %ctor
```

- **Instance fields** are initialised *inside* the constructor function, right after `this` is available. For a base class that's at entry. For a derived class it's right after `super(...)` returns. `buildFunction` takes an extra "field initialisers" list for constructors, and emits `StoreProperty %this, <init> "f"` for each.
- **Methods** are built with `buildFunction` (they have a receiver, so `isArrow = false`).
- **`super(args)`** in a derived constructor: `%this = CallBuiltin "superConstruct" %super, args…`, then `store("this", %this)`. That's why `this` is an ordinary `Let` binding and not a raw parameter.
- **`super.m(args)`**: `%hp = LoadProperty %superProto "m"; CallProperty %hp, %this, args…`. The method needs its *home object*, so pass the superclass prototype as a captured variable named `"%home"`, using the same trick as lexical `this`.
- **Modifier keywords** aren't values. Inspect them with `dynamic_cast<StaticKeyword*>(mod.get())` while lowering, and never visit them. `visitPublicKeyword`, `visitPrivateKeyword`, `visitProtectedKeyword` and `visitStaticKeyword` should call `unsupported("modifier used as expression")`. `private` and `protected` are compile-time access checks with no runtime IR.
- `visitClassExpression` gives the same lowering with the class value returned instead of declared. `visitMethodDefinition` is only reached through class lowering, so a direct visit is a bug.

14.6 `visitTry` and `visitCatch`

AST: `TryStatement{ block, handler (CatchClause{ param, body }), finalizer }`.

Today: stubs. Exceptions are not modelled.

Design. Exceptions add *implicit* edges: **any** instruction inside the `try` block that can throw may transfer control to the handler. Drawing an edge from every instruction would make the CFG explode, so use the standard compact form:

1. **Regions:** every block built while the `try` block is open records its handler (`BasicBlock::handler`, a new field). Turbine turns contiguous ranges into a *handler table* `(startPc, endPc) → handlerPc`, as V8 does. On a throw, DeathStar looks up the table for the current frame, jumps to the handler, and puts the exception in the accumulator.
2. **Catch entry:** the handler block starts with `%e = LandingPad`, which reads the accumulator, and `declare(param, %e, Let)` in the catch scope.
3. **SSA across the implicit edges.** The catch block can be entered from *any point* in the `try`, so the value of an SSA variable assigned in the `try` is unknown at catch entry. A precise Phi would need one operand per throwing instruction. The simple, robust fix is to **demote** such variables to frame slots:
    - Before building a function, collect the names **assigned inside any `try` block** (the resolver can do this).

- For those names, `declare` emits `Store name` (`IROp::Store`, `label = name`), `lookup` emits `%v = Load name`, and `store` emits `Store name`. They behave like context variables, but in the frame, with no heap allocation.
- The catch block then needs no Phis: demoted variables are re-read from their slots, and every other variable has the same value it had at `try` entry, because the `try` never assigned it.

4. **`finally`:** duplicate the finalizer code on every exit path:
   - Normal completion of the `try` or the `catch`: emit the finalizer, then `Jump` to the end.
   - Exceptional exit (an exception in the `catch`, or no `catch` at all): give the region a second handler block, `%e2 = LandingPad; <finalizer>; Throw %e2`.
   - `return`, `break` or `continue` crossing the `finally`: keep a `finallyStack`. `visitReturn`, `visitBreak` and `visitContinue` emit (inline) every pending finalizer between the current point and their target before emitting their terminator.

```
R IRBuilderVisitor::visitTry(TryStatement* t) {
    BasicBlock* endB     = createBlock("try.end");
    BasicBlock* handlerB = t->handler ? createBlock("catch") : nullptr;

    vector<Scope> before = scopes;
    vector<vector<Scope>> endIn;

    handlerStack.push_back(handlerB);                  // createBlock() stamps ->handler on new
blocks
    if (t->finalizer) finallyStack.push_back(t->finalizer.get());
    BasicBlock* tryB = createBlock("try");
    emitJump(tryB); currentBlock = tryB;
    t->block->accept(*this);
    handlerStack.pop_back();

    if (!isTerminated()) {
        if (t->finalizer) t->finalizer->accept(*this);  // normal completion
        endIn.push_back(scopes); emitJump(endB);
    }

    if (handlerB) {
        currentBlock = handlerB; scopes = before;       // demoted vars are re-read from slots
        scopes.push_back({ Scope::Type::Block, nullptr, {} });
        IRInstruction lp(IROp::LandingPad, createTemp(IRType::Any), {});
        emit(lp);
        if (!t->handler->param.empty()) declare(t->handler->param, lp.result,
BindingKind::Let);
        t->handler->body->accept(*this);
        scopes.pop_back();
        if (!isTerminated()) {
            if (t->finalizer) t->finalizer->accept(*this);
            endIn.push_back(scopes); emitJump(endB);
        }
    }
    if (t->finalizer) finallyStack.pop_back();
    // (exceptional-path finalizer + rethrow handler omitted for brevity: see step 4)

    currentBlock = endB;
    if (endIn.empty()) { scopes = before; return true; }
    mergeScopes(endB, endIn);
    return true;
}

R IRBuilderVisitor::visitCatch(CatchClause*) { unsupported("catch clause outside try"); }
```

The handler block has no *explicit* predecessors, because it's reached through the handler table. The reachability sweep in §16 must treat every block listed as a handler as a root, or it will delete the catch code.

14.7 `visitImportDeclaration`

AST: `path` (a string-literal token, quoted) and `sourceFile` (the importing file).

Interpreter semantics: it resolves `path` relative to the importer's directory, parses the file, and **executes its statements in the current environment**. In effect, it's a textual include.

Recommended: do the same, but as an **AST pre-pass** before the resolver runs, so that capture analysis sees the imported code:

```
// before resolver.resolveProgram(program):
void expandImports(vector<unique_ptr<Statement>>& program, unordered_set<string>& seen) {
    vector<unique_ptr<Statement>> out;
    for (auto& s : program) {
        if (auto* imp = dynamic_cast<ImportDeclaration*>(s.get())) {
            string path = resolveImportPath(imp->sourceFile, imp->path.lexeme);  // strip
quotes, weakly_canonical
            if (!seen.insert(path).second) continue;                              // import
each file once (cycles)
            auto ast = get_ast(read_file(path), path);
            expandImports(ast, seen);
            for (auto& x : ast) out.push_back(std::move(x));
        } else out.push_back(std::move(s));
    }
    program = std::move(out);
}
R IRBuilderVisitor::visitImportDeclaration(ImportDeclaration*) { unsupported("import not
expanded"); }
```

A real module system, with its own scope, `export` and live bindings, would compile each file to its own `IRFunction` that returns a namespace object. That's future work, and today's language doesn't need it.

14.8 `visitEnumDeclaration`

AST: `name` and `members` (each has `name`, an optional `value`, and `computedValue`).

Interpreter: a no-op (`return true`), so enums currently have no runtime value. The IR builder can match that (keep `return true`) or **define** the semantics. The natural definition is a frozen object:

```
R IRBuilderVisitor::visitEnumDeclaration(EnumDeclaration* e) {
    auto obj = emitValue(IROp::NewObject, IRType::Object);
    for (auto& m : e->members) {
        auto v = m.value ? value(m.value.get()) : emitNumber(m.computedValue);
        IRInstruction st(IROp::StoreProperty, nullptr, { obj, v });
        st.immediate = m.name; emit(st);
    }
    declare(e->name, obj, BindingKind::Const);
    return true;
}
```

Whichever you choose, keep the interpreter and the IR path in agreement. Differential testing (§17) depends on it.

14.9 `visitAwaitExpression` and `visitYieldExpression`

Today: stubs.

Both **suspend** a running function and resume it later, keeping its locals. There are two standard implementations:

Approach	IR impact	VM impact
State-machine transform (regenerator style)	The function is rewritten into a `switch` over a resume-state number. Live values are demoted to heap slots, as for `try` (§14.6).	none
Heap frames with VM suspension	New terminator-like ops `Suspend %value` / `%v = Resume`. Values live across a suspension in frame registers that the VM keeps alive.	`CallFrame` must be heap-allocated and resumable. Today `DeathStar::run` recurses on the C++ stack.

Neither is small. Until one is chosen, both visitors should call `unsupported("await")` / `unsupported("yield")`. `is_async` on functions should also be rejected, rather than compiling an async function as if it were synchronous.

14.10 `visitUIExpression`

`UIViewExpression` (`name`, `viewType`, …) builds native UI through the platform layer (`GUI/`, Cocoa on macOS). It has no meaning in a portable IR beyond "call the platform". If it's supported at all, lower it to `CallBuiltin "ui.<viewType>"` with the evaluated attributes, and keep the work in the runtime. Otherwise call `unsupported("UI view expression")`.

Part IV — Building it out

15. Recommended order of work

Each step is testable on its own with the tools in §16 and §17.

Step	Builder work (this guide)	Downstream prerequisites
1	CFG helpers (§6.2), `value()` / `unsupported()`, the IR printer and verifier (§16)	–
2	Fix `store` (§7.4, IR1), `runtimeDepth` (IR4, IR5), and implicit return (IR9)	–
3	`visitIf` with `Jump` + `mergeScopes` (§12.4)	Turbine: `Jump`/`If` lowering with patched offsets (T1), critical-edge splitting. VM: `kJump`, `kJumpIfToBooleanFalse`
4	Loops: `while`, `do`, `for`, `break`, `continue` (§12.6–12.10)	Turbine: fix `resolvePhis` (T6)
5	Literals cleanup (§11.3–11.6), unary, update, short-circuit, conditional, assignment (§13.1–13.6)	Turbine T2, T3, T4. VM V1, V2, V3, plus comparison and arithmetic bytecodes
6	Resolver pre-pass (§9.3), top level as a function (N6), hoisting (N7), parameters (IR2), closure context (N4)	VM: keep call arguments (V4), `Parameter` lowering, `kCreateClosure` capturing the current context
7	Calls with receivers, members, `new`, arrays, objects, templates (§13.8–13.13)	Turbine/VM: property and element bytecodes, `CallProperty`, `Construct`, `CallBuiltin`
8	`switch`, `for-in`/`for-of` (fix N5 first by making it `unsupported`), `throw`	VM: iterator builtins, `kThrow`
9	Functions as expressions, arrows, `this`, classes (§14.2–14.5)	–
10	`try/catch/finally` (§14.6)	handler table in Turbine and VM
11	Pharoah: trivial-Phi removal, side-effect-aware DCE, then constant folding	–

16. Tooling: printer, verifier, cleanup

16.1 IR printer (`--dump-ir`)

```
std::string opName(IROp op);   // a switch over all enumerators

void printIR(const IRModule& m, std::ostream& os) {
    for (size_t fi = 0; fi < m.functions.size(); fi++) {
        auto& f = *m.functions[fi];
        os << "function #" << fi << " '" << f.name << "'" << (f.entry_point ? "  [entry]" : "")
<< "\n";
        for (auto& b : f.blocks) {
            os << "  " << b->name << ":";
            if (!b->predecessors.empty()) {
```

```
                os << "  preds=[";
                for (size_t i = 0; i < b->predecessors.size(); i++)
                    os << (i ? ", " : "") << b->predecessors[i]->name;
                os << "]";
            }
            os << "\n";
            for (auto& in : b->instructions) {
                os << "    ";
                if (in.result) os << in.result->name << " = ";
                os << opName(in.op);
                for (size_t i = 0; i < in.operands.size(); i++)
                    os << (i ? ", " : " ") << (in.operands[i] ? in.operands[i]->name : "
<null>");
                if (in.op == IROp::Constant)       os << " " << std::get<int>(in.immediate);
                if (in.op == IROp::HeapNumber)     os << " " << std::get<double>(in.immediate);
                if (in.op == IROp::StringConstant || in.op == IROp::LoadProperty || in.op ==
IROp::StoreProperty)
                    os << " \"" << std::get<std::string>(in.immediate) << "\"";
                if (in.contextSlot >= 0) os << " slot=" << in.contextSlot;
                if (in.op == IROp::LoadContextSlot || in.op == IROp::StoreContextSlot)
                    os << " depth=" << in.contextDepth;
                if (in.childFunction) os << " fn=" << in.childFunction->name;
                if (!in.targets.empty()) {
                    os << " ->";
                    for (size_t i = 0; i < in.targets.size(); i++)
                        os << (i ? ", " : " ") << in.targets[i]->name;
                }
                if (!in.label.empty()) os << "    ; " << in.label;
                os << "\n";
            }
        }
    }
}
```

Wire it into `main.cpp` as a `--dump-ir <file>` flag, next to `--i` and `--c`. That's the single most useful debugging tool for every step in §15.

16.2 Verifier

Run it after `build()`, and after every Pharoah pass in debug builds. It checks the contract from §5:

```
void verifyFunction(const IRFunction& f) {
    auto fail = [&](const BasicBlock* b, const string& msg) {
        throw std::logic_error("IR verify: " + f.name + "/" + (b ? b->name : "?") + ": " +
msg);
    };
    std::unordered_set<const IRValue*> defined;
    for (auto& b : f.blocks)
        for (auto& in : b->instructions) if (in.result) {
            if (!defined.insert(in.result.get()).second) fail(b.get(), "value defined twice: "
+ in.result->name);
        }

    for (auto& bp : f.blocks) {
        const BasicBlock* b = bp.get();
        if (b->instructions.empty()) fail(b, "empty block (C1)");
        bool inPhis = true;
        for (size_t i = 0; i < b->instructions.size(); i++) {
            auto& in = b->instructions[i];
            bool last = (i + 1 == b->instructions.size());
            bool term = in.op == IROp::If || in.op == IROp::Jump || in.op == IROp::Return ||
in.op == IROp::Throw;
```

```
            if (term != last) fail(b, last ? "missing terminator (C1)" : "terminator mid-block
(C1)");
            if (in.op == IROp::Phi) {
                if (!inPhis) fail(b, "Phi after non-Phi (C4)");
                if (in.operands.size() != b->predecessors.size()) fail(b, "Phi arity != #preds
(C4)");
            } else inPhis = false;
            for (auto& op : in.operands)
                if (!op || !defined.count(op.get())) fail(b, "use of undefined value (C5/C6)");
            if (term && in.targets.size() != b->successors.size()) fail(b, "targets !=
successors (C3)");
        }
        for (auto* s : b->successors) {
            auto n1 = std::count(b->successors.begin(), b->successors.end(), s);
            auto n2 = std::count(s->predecessors.begin(), s->predecessors.end(), b);
            if (n1 != n2) fail(b, "succ/pred mismatch with " + s->name + " (C2)");
        }
    }
}
```

The verifier checks "defined somewhere in the function", not dominance. A full dominance check (C5) needs a dominator tree. Add one when Pharoah needs dominators anyway.

16.3 Unreachable-block removal

```
void removeUnreachableBlocks(IRFunction& f) {
    std::unordered_set<BasicBlock*> live;
    std::vector<BasicBlock*> work = { f.blocks.front().get() };
    for (auto& b : f.blocks) if (isHandlerBlock(b.get())) work.push_back(b.get());   // §14.6
roots
    while (!work.empty()) {
        auto* b = work.back(); work.pop_back();
        if (!live.insert(b).second) continue;
        for (auto* s : b->successors) work.push_back(s);
    }
    for (auto& b : f.blocks) {
        if (live.count(b.get())) continue;
        for (auto* s : b->successors) {                          // drop the edge AND its Phi
operand
            auto& preds = s->predecessors;
            for (size_t k = 0; k < preds.size(); ) {
                if (preds[k] != b.get()) { k++; continue; }
                for (auto& in : s->instructions)
                    if (in.op == IROp::Phi) in.operands.erase(in.operands.begin() + k);
                preds.erase(preds.begin() + k);
            }
        }
    }
    f.blocks.erase(std::remove_if(f.blocks.begin(), f.blocks.end(),
                   [&](auto& b){ return !live.count(b.get()); }), f.blocks.end());
}
```

Run trivial-Phi removal (§8.5) afterwards: dropping operands often leaves Phis with a single operand.

17. Testing strategy

1. **Golden IR tests.** Write one small `.ardan` file per construct (`if-assign.ardan`, `while-counter.ardan`, `closure-counter.ardan`, …), each paired with an `.ir` file holding the expected `--dump-ir` output. Sorted slot order (§9.4) and the result-after-operands numbering (§13.1) keep these stable.
2. **Verifier always on.** Every golden test also runs `verifyFunction` on every function. A contract violation fails the test even when the output looks plausible.
3. **Differential testing.** Run each program from `tests/*.ardan` with `--i` (the tree-walking interpreter) and through the IR pipeline, and compare stdout. The interpreter is the reference semantics. Any divergence is a bug in the builder, Turbine or DeathStar, unless it's a documented intentional difference such as enums (§14.8) or per-iteration `let` (§10.4).
4. **Minimal reproductions.** Every row in `09-known-issues.md` already has a minimal program. Turn each one into a regression test the moment it's fixed.

A starter set, one program per feature:

```
// if-assign
let x = 1; if (x < 2) { x = 5; } print(x);                    // 5
// if-else-phi
let y = 0; if (y) { y = 1; } else { y = 2; } print(y);        // 2
// while-counter
let i = 0; while (i < 3) { i = i + 1; } print(i);             // 3
// break-continue
let s = 0; for (let k = 0; k < 10; k++) { if (k == 5) { break; } if (k % 2) { continue; } s +=
k; } print(s); // 6
// short-circuit
let c = 0; function inc() { c = c + 1; return true; } let r = false && inc(); print(c);   // 0
// closure-counter
function counter(n) { return function () { n = n + 1; return n; }; } let k2 = counter(10);
k2(); print(k2()); // 12
// recursion (N7)
function fact(n) { if (n < 2) { return 1; } return n * fact(n - 1); } print(fact(5));     //
120
// top-level capture (N6)
let base = 100; function add(v) { return base + v; } print(add(1));                       //
101
```

18. Worked example: one program, start to finish (proposed builder)

```
let total = 0;
function addAll(limit) {
    let i = 0;
    while (i < limit) {
        if (i % 2 == 0) { total += i; }
        i++;
    }
    return total;
}
print(addAll(5));       // 0 + 2 + 4 = 6
```

Resolver. The top level declares `total` and `addAll`. `addAll` declares `limit`, `i` and `this`. `total` is referenced from `addAll`, so the top level captures `total` and gets one slot (`total → 0`). `addAll` captures nothing, so `i` and `limit` stay SSA.

IR (proposed). Numbering follows the recommended `build()` order: `CreateContext`, then hoisted functions, then the remaining statements.

```
function #0 'addAll'
  addAll1:
    %1 = Parameter 0                          ; this
    %2 = Parameter 1                          ; limit
    %3 = Zero                                 ; i = 0
    Jump while.header2
  while.header2:  preds=[addAll1, if.end6]
    %4 = Phi %1, %4                           ; this   (trivial: removed by §8.5)
    %5 = Phi %2, %5                           ; limit  (trivial: removed)
    %6 = Phi %3, %16                          ; i
    %7 = LessThan %6, %5
    If %7 -> while.body3, while.exit4
  while.body3:  preds=[while.header2]
    %8 = Constant 2
    %9 = Modulo %6, %8
    %10 = Zero
    %11 = Equal %9, %10
    If %11 -> if.then5, if.end6
  if.then5:  preds=[while.body3]
    %12 = LoadContextSlot slot=0 depth=0     ; total (in <main>'s context; addAll owns none)
    %13 = Add %12, %6
    StoreContextSlot %13 slot=0 depth=0
    Jump if.end6
  if.end6:  preds=[while.body3, if.then5]   ; no Phis: total lives in memory, i is unchanged
here
    %14 = ToNumber %6                         ; i++
    %15 = Constant 1
    %16 = Add %14, %15
    Jump while.header2
  while.exit4:  preds=[while.header2]
    %17 = LoadContextSlot slot=0 depth=0
    Return %17
function #1 '<main>'  [entry]
  entry0:
    %0 = CreateContext slot=1                 ; slots: total → 0
    %18 = Closure addAll                      ; hoisted; captures <main>'s context implicitly
    %19 = Zero
    StoreCurrentContextSlot %19 slot=0        ; let total = 0 (captured)
    %20 = Constant 5
    %21 = Call %18, %20
    Print %21
    %22 = Undefined                           ; value of the print(...) expression (N1 fix),
dead
    %23 = Undefined
    Return %23
```

What each layer contributes

- **Phis** exist only for `i`, which is SSA and modified in the loop, plus the trivial ones that cleanup removes. `total` never needs a Phi because it lives in a context slot.
- **`if.end6` has no Phi.** `i` isn't assigned inside the `if`, and `total` is memory.
- **`runtimeDepth`:** `addAll` owns no context, so the top level's context is builder depth 0, which is runtime depth 0 from `parentContext`. The `Closure` captured `<main>`'s own context, which becomes `addAll`'s `parentContext`.
- **Turbine** lowers each Phi to copies at the end of each predecessor: `%3 → %6` in `addAll1`, and `%16 → %6` in `if.end6`. Then each `If`/`Jump` becomes a jump bytecode with a patched offset.

Appendix A — Status of every `visit*` method

Method	Today	Section
`build`	works for straight-line code; null `Return`, no top-level context (N6)	§11.1
`visitExpression`	works; leaks `R`	§11.2
`visitBlock`	correct	§12.1
`visitVariable`	works; `let x;` is an undefined value	§12.3
`visitIf`	CFG without jumps; Phis never form (IR1)	§12.4
`visitWhile`	broken CFG (IR6, N2)	§12.7
`visitFor`	broken CFG (IR7); `for(;;)` crashes	§12.9
`visitDoWhile`	stub	§12.8
`visitReturn`	works; no operand without an argument (T7)	§12.5
`visitBreak` / `visitContinue`	stubs	§12.10
`visitSwitch` / `visitSwitchCase`	stubs	§12.11
`visitForIn`	**crashes** (N5)	§12.12
`visitForOf`	stub	§12.12
`visitThrow`	stub	§12.13
`visitTry` / `visitCatch`	stubs	§14.6
`visitFunction`	works for parameterless closures; IR2, IR8, IR9, N4, N7	§14.1
`visitFunctionExpression` / `visitArrowFunction`	stubs	§14.2–14.3
`visitClass` / `visitClassExpression` / `visitMethodDefinition` / `visitSuper`	stubs	§14.5
`visitImportDeclaration`	stub (no-op)	§14.7
`visitEnumDeclaration`	no-op (matches the interpreter)	§14.8
`visitInterfaceDeclaration` / `visitEmpty`	correct no-ops	§12.2
`visitNumericLiteral`	works; immediate type not canonical (T3)	§11.3
`visitStringLiteral`	works; fake operand	§11.4
`visitTrue/False/Null/UndefinedKeyword`	work; fake operands on True/False	§11.6
`visitIdentifier`	works	§11.5
`visitBinary`	arithmetic/comparison work; `&&` / `||` / `??` eager (N3); assignment returns no value (IR10, IR11)	§13.1–13.3
`visitAssignment` / `visitLogical`	unreachable stubs	§13.2–13.3

Method	Today	Section
`visitUnary` / `visitUpdate`	stubs (IR3)	§13.4–13.5
`visitConditional`	stub	§13.6
`visitSequence` / `visitComma`	stubs	§13.7
`visitCall`	works for plain calls; `print` gives no value (N1); no methods or spread	§13.8
`visitMember` / `visitNew` / `visitArray` / `visitObject` / `visitTemplateLiteral` / `visitSpreadExpression`	stubs	§13.9–13.13
`visitThis`	stub	§14.3
`visitRestParameter` / modifier keywords / `visitLiteral` / `visitProperty`	stubs (should be `unsupported`)	§14.4, §14.5, §11.7
`visitAwaitExpression` / `visitYieldExpression` / `visitUIExpression`	stubs	§14.9–14.10

Appendix B — Issues found by this guide

These add to the list in `docs/compiler/09-known-issues.md`.

#	Issue	Repro	Section
N1	`print(...)` returns `true` instead of a value	`let r = print(1);` → `bad_variant_access`	§13.8
N2	after a loop, `scopes` holds latch values instead of header Phis (shows up once IR1 is fixed)	`let i = 0; while (i < 0) { i = 9; }` `print(i);` → reads a value whose definition never ran	§8.4, §12.7
N3	`&&`, `||` and `??` evaluate both operands	`false && f()` calls `f`	§13.1–13.2
N4	a `Closure` built in a non-owning function passes an *outer* function's context register	3-level nesting where only the outermost function owns a context	§10.3
N5	`visitForIn` leaks a scope and leaves an empty orphan block, so Turbine underflows	`for (let k in o) { }`	§12.12
N6	top-level bindings can't be read from functions (no top-level context)	`let a = 1; function f(){ return a; }` `f();` → `Undefined variable`	§11.1
N7	no hoisting or self-reference: calling a function before its declaration, or recursively, fails	`function f(n){ return n ? f(n-1) : 0; }`	§14.1
N8	`break`/`continue` targets (once added) must not leak into nested functions	–	§14.1
N9	the member-assignment evaluation order would be RHS before object/key	`o()[k()] = v()`	§13.3
N10	`walkForFreeVars` treats a name as free if *any* bound set lacks it (should be *none* has it)	latent; shows up with nested block levels	§9.2

Appendix C — Further reading

- M. Braun, S. Buchwald, S. Hack, R. Leißa, C. Mallon, A. Zwinkau. *Simple and Efficient Construction of Static Single Assignment Form.* CC 2013. On-the-fly SSA, incomplete Phis and trivial-Phi removal: the algorithm behind §8.
- R. Cytron et al. *Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.* TOPLAS 1991. The classic dominance-frontier algorithm.
- P. Briggs et al. *Practical Improvements to the Construction and Destruction of SSA Form.* 1998. Phi elimination, the lost-copy and swap problems, and critical edges (§12.4).
- V8 design docs: *Ignition* (the accumulator bytecode Turbine targets), *context allocation and closures*, *TurboFan sea of nodes* (the design the reserved opcodes come from).
- `docs/compiler/` in this repository: the model (03), builder (04), Pharoah (05), Turbine (06), DeathStar (07), worked example (08), known issues (09).